Distributional Learning of Context-Free Languages under Fixed Finite-Monoid Typing

Takayuki Kuriyama
Independent Researcher
growup.kuriyama@gmail.com

Abstract

We study positive-data learning of context-free languages under a fixed homomorphism $h : \Sigma^* \to M$ to a finite monoid. For each fixed h, a polynomial-time set-driven operator reconstructs context-free targets substitutable relative to h from finite positive witnesses, while a separate conservative learner identifies all nonempty targets in Gold's sense.

Characteristic-data bounds depend on shortest yields in type-refined grammars. For a two-element semilattice typing, we construct finite targets of grammar size and ordinary thickness $O(n)$ that require exponentially long positive characteristic data for every set-driven operator admitting characteristic samples throughout the fixed-typing class. The same family forces exponential typed thickness for every reduced grammar presentation in our start-separated binary normal form, regardless of its size.

Polynomial characteristic-data bounds in grammar size and ordinary thickness hold for typings determined by a fixed prefix–suffix window together with a finite-group value; for every fixed h, linear targets admit characteristic data polynomial in linear grammar size. The classical locally trivial-semigroup criterion characterizes fixed-window-determined typings. Nonregular linear and nonlinear targets occur beyond fixed windows, whereas the one-bracket Dyck language lies outside every fixed-typing class.

Keywords: Grammatical inference; Context-free languages; Distributional learning; Positive-data identification; Finite monoids

1 Introduction

1.1 Background and motivation

Gold [1] showed that no language class containing all finite languages and at least one infinite language is identifiable in the limit from positive data alone. For context-free languages, distributional learning recovers positive results by restricting which substrings may be compared across contexts; classical examples are Clark–Eyraud substitutability [2] and Yoshinaka's (k, ℓ)-substitutability [3]. See de la Higuera [4] for general background and Kanazawa [5, 6] for positive-data learning of categorial grammars. In particular, Kanazawa's monograph develops characteristic samples for rigid grammars and finite-elasticity arguments for the string languages of k-valued categorial grammars [6, Chapters 5 and 8]. This is a complementary line of research: here the learning bias instead fixes a finite-monoid typing of terminal factors and requires distributional substitutability.

Relation to earlier work and general signature learning. This paper substantially corrects and extends the author's 2014 preprint (arXiv v3) [7], which introduced substitutability relative to recognizable equivalence relations and proposed positive-data learning in that setting. The 2014 preprint predates the publication of Yoshinaka's general framework of signatures with disjoint sorts [8]. In Yoshinaka's framework, for the finite-monoid homomorphism $h : \Sigma^* \to M$

considered in this paper, the nonempty fibers $h^{-1}(m) \cap \Sigma^+$ can be represented as disjoint sorts, with typed concatenation and an operation promoting a typed factor to the whole-string sort. The resulting signature has the same substitutability condition on nonempty factors as the fixed-h definition below. Consequently, the *existence* of a positive-data learner for a fixed h is also covered by the qualitative signature-based result of [8, Theorem 1]; our particular batch and sequential constructions are analyzed directly here. The finite-monoid-specific contributions are retyping closure, encoded characteristic-data bounds (rather than merely witness cardinality), their extension to window–finite-group typings, and the linear and expressive-boundary results. The locally trivial semigroup criterion itself is classical. The broader distributional learning setting is surveyed by Clark and Yoshinaka [9]; local substitutability is studied by Coste, Garet, and Nicolas [10] and Coste and Nicolas [11].

We study a recognizable-congruence version of this program. Let $h : \Sigma^* \to M$ be a homomorphism into a finite monoid and $\sim_h := \ker h$. For $x \in \Sigma^*$, write $\mathcal{D}_L(x) := \{(u, v) : uxv \in L\}$. A language L is $\sim_h$-*substitutable* if, for nonempty x, y, $h(x) = h(y)$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ imply $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. Write $\mathcal{C}_h^{\mathrm{cf}}$ for the context-free $\sim_h$-substitutable languages, and call this the *fixed-h slice*. Thus full distributional equality is required only after a fixed finite algebraic comparison.

For a minimal example, take $\Sigma = \{a\}$ and $L = \{a, aa\}$. Although $(\lambda, \lambda) \in \mathcal{D}_L(a) \cap \mathcal{D}_L(aa)$, the context (a, λ) belongs to $\mathcal{D}_L(a)$ but not to $\mathcal{D}_L(aa)$. Thus L is not Clark–Eyraud substitutable. Under the parity morphism $h(a^n) = n \bmod 2$ into $\mathbb{Z}/2\mathbb{Z}$, however, a and aa have distinct types and $L \in \mathcal{C}_h^{\mathrm{cf}}$.

Finite-index monoid congruences, equivalently kernels of morphisms into finite monoids, provide finite typings. For each fixed h, write RS_h for the class of all $\sim_h$-substitutable languages, without a context-freeness restriction. For expressive comparison, let $\mathsf{RS} := \bigcup_h \mathsf{RS}_h$, where h ranges over finite-monoid homomorphisms on Σ^*. Every regular language belongs to this union. Our learning results, however, concern only $\mathcal{C}_h^{\mathrm{cf}}$ for one fixed h; no learner is claimed for RS.

Unlike one fixed prefix–suffix window, h can combine independent finite-state factor features by a product. If the finite typing is allowed to be refined accordingly, the framework is stable under regular filtering and arbitrary inverse homomorphisms, including erasing ones (Proposition 3.2). This is a change-of-typing statement, not closure of one fixed-h slice. An algebraic criterion identifies the typings determined by fixed prefix–suffix windows on nonempty factors (Proposition 3.5); combining windows with finite groups yields a broader characteristic-data bound (Corollary 7.5). Examples below exhibit nonregular linear and nonlinear targets outside every fixed-window class, while counter and Dyck obstructions delimit the reach of fixed finite-monoid typing.

Specifying h is a prior choice, related to typing or domain bias in automata inference [12]; here types apply to entire terminal factors, not to occurrences with their outer contexts. For linear grammars, yield typing also induces a regular control set of production-label sequences, giving a limited but concrete connection with Takada's regular control-set approach [13, 14] (Section 8), without an equivalence claim.

1.2 Contributions and main results

The qualitative existence of a positive-data learner also follows from Yoshinaka's general many-sorted signature framework. Here we give a direct construction for context-free targets under a fixed finite-monoid typing and prove its correctness. Specifically, the set-driven operator Rec_h reconstructs every target in $\mathcal{C}_h^{\mathrm{cf}}$ exactly from finitely many canonical positive witnesses. A separate conservative sequential learner Learn_h identifies every nonempty target in Gold's sense; reconstruction and each update take time polynomial in the accumulated sample size.

Building on this learning construction, we emphasize two main contributions.

(1) *Quantitative guarantees.* For arbitrary fixed h, we bound the encoded size of character-

istic data in terms of grammar size and the shortest terminal yields after splitting target nonterminals according to their h-values. When a fixed prefix–suffix window together with a finite-group value determines the h-type, that is, $\ker(h_{k,\ell} \times g)|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$, ordinary source-grammar thickness suffices. This recovers Yoshinaka's fixed-window regime and extends the bound to finite-group refinements. Linear fixed-h targets admit characteristic data polynomial in the size of a linear CFG. We also show that refining the typing can force exponentially long characteristic data for the reconstruction operator, even for finite Clark–Eyraud substitutable targets; a second target in the same slice extends the characteristic-sample lower bound to every set-driven operator admitting characteristic samples for both targets (Proposition 6.3). Moreover, Corollary 6.4 gives an exponential typed-thickness lower bound across all reduced presentations of these targets in our start-separated binary normal form, without a size restriction.

(2) *Structural and expressive position.* We establish closure properties obtained by refinement and products of finite typings (not closure of an individual fixed-h slice), record the classical algebraic criterion for typings determined by fixed prefix–suffix windows, and relate linear yield typing to regular control sets. Explicit nonregular linear and nonlinear examples lie beyond all fixed-window classes; counter/Dyck obstructions and comparison with Clark's congruential family delimit the framework.

1.3 Organization of the paper

Sections 2–3 set up the framework, Sections 4–5 prove learnability, Sections 6–8 give the quantitative results, and Section 9 gives the expressiveness comparisons. Long normalization proofs are deferred to the appendices.

2 Preliminaries

Let Σ be a finite alphabet, let Σ^* be the free monoid with identity λ, and put $\Sigma^+ := \Sigma^* \setminus \{\lambda\}$. A pair $(u, v) \in \Sigma^* \times \Sigma^*$ is called a *terminal context*. For $L \subseteq \Sigma^*$ and $x \in \Sigma^*$, the *distribution* of x in L is $\mathcal{D}_L(x) := \{(u, v) \in \Sigma^* \times \Sigma^* : uxv \in L\}$. The *syntactic congruence* of L is defined by $x \equiv_L y$ iff $\mathcal{D}_L(x) = \mathcal{D}_L(y)$, for $x, y \in \Sigma^*$ [15, Chapter III, Proposition 10.2]. As usual, $\equiv_L$ is a monoid congruence: if $x \equiv_L x'$ and $y \equiv_L y'$, then $xy \equiv_L x'y'$.

For a word $w \in \Sigma^*$ and a letter $a \in \Sigma$, let $|w|_a$ denote the number of occurrences of a in w.

A context-free grammar (CFG) is a tuple $G = (V, \Sigma, P, S)$ with finite nonterminal set V, start symbol $S \in V$, and productions $A \to \alpha$ with $A \in V$ and $\alpha \in (V \cup \Sigma)^*$. A *sentential form* is any word in $(V \cup \Sigma)^*$. We write $\alpha \Rightarrow_G \beta$ for one derivation step in G, and $\Rightarrow_G^*$ for its reflexive-transitive closure. Thus $L(G) := \{w \in \Sigma^* : S \Rightarrow_G^* w\}$. For $A \in V$, put $L_G(A) := \{w \in \Sigma^* : A \Rightarrow_G^* w\}$. For a nonterminal $X \in V$, a terminal context (u, v) is a *terminal reaching context of X in G* if $S \Rightarrow_G^* uXv$. A nonterminal is *reachable* if it occurs in a sentential form reachable from S, *productive* if it derives a terminal word, *useless* if it is not both, and *nullable* if it derives λ; G is *reduced* if it has no useless nonterminal. A *unit production* (*unit rule*) is $A \to B$ with $A, B \in V$, and the *unit closure* is the reflexive-transitive closure of the induced relation on V. For $L(G) \neq \emptyset$, the *trim* (*reduced part*) of G deletes the nonproductive nonterminals and then the unreachable ones, each with its incident productions. A CFG is *linear* if every right-hand side contains at most one nonterminal. By *binarization* we mean replacement of longer right-hand sides by equivalent chains of binary productions using fresh nonterminals. We write CFL for the class of context-free languages. For any finite representation $\mathcal{R}$, $|\mathcal{R}|$ denotes the length of a fixed reasonable encoding; for a grammar this is polynomially equivalent to the usual symbol count of its productions. Following the thickness parameter used by Wakatsuki–Tomita [16] and adopted by Yoshinaka [3] (see also Eyraud–Heinz–Yoshinaka [17,

Definition 2.11]), for a reduced CFG $G = (V, \Sigma, P, S)$ we use the shortest-yield *thickness*

$$\tau_G := \max_{A \in V} \min\{\, |w| : A \Rightarrow_G^* w,\ w \in \Sigma^* \,\}.$$

2.1 Learning from positive data

We use Gold's explanatory model [1]. Let H_0 denote a fixed CFG with one start symbol and no productions, so $L(H_0) = \emptyset$. We use texts only for nonempty targets and represent the empty language separately by H_0; equivalently one may adjoin the usual pause symbol. For a nonempty language L, a *text* is an infinite sequence $\mathcal{T} = (w_1, w_2, \ldots)$ over L in which every word of L occurs. Write $\mathcal{T}[n] = (w_1, \ldots, w_n)$ and $K_n := \{w_1, \ldots, w_n\}$. A learner Learn *identifies a nonempty L in the limit* if for every text $\mathcal{T}$ for L there are n_0 and a grammar H such that $\mathsf{Learn}(\mathcal{T}[n]) = H$ for every $n \geq n_0$ and $L(H) = L$. Thus the hypothesis representation itself eventually stabilizes. A sequential learner is *conservative* if it changes its current hypothesis only when the newly presented positive datum is not generated by that hypothesis.

We distinguish this sequential learner from a *set-driven reconstruction operator* Rec: this is a map from finite sets of words to grammars; in particular, its output is independent of presentation order and multiplicity. We call Rec *sample-consistent* if $K \subseteq L(\mathsf{Rec}(K))$ for every finite sample K. A finite set $\mathcal{F} \subseteq L$ is a *characteristic sample of L for* Rec if $\mathcal{F} \subseteq K \subseteq L$ implies $L(\mathsf{Rec}(K)) = L$ for every finite K; this is the positive-data, set-driven specialization of the characteristic samples of de la Higuera [18] (cf. [4, Section 7.3.3, Definition 7.3.3], formulated for sequential presentations). We measure sample size by $\|K\| := \sum_{w \in K}(|w| + 1)$, so that λ counts and $|K| \leq \|K\|$, and call $\|\mathcal{F}\|$ the *characteristic data* of $\mathcal{F}$. Polynomial update time and polynomial characteristic data are distinct criteria [4, Sections 7.3.2–7.3.3]. In the terms of Eyraud–Heinz–Yoshinaka [17, Definitions 2.5 and 2.8], our polynomial characteristic-data guarantees are *set-based* (correct reconstruction by Rec_h from every finite sample containing a characteristic sample); the *incremental* criterion, a correct and stable hypothesis of Learn_h as soon as such a sample has appeared, is not claimed. Gold stabilization of Learn_h is proved in Corollary 5.5.

2.2 Fixed finite-monoid typing

Fix a finite monoid M and a homomorphism $h : \Sigma^* \to M$, represented by the multiplication table of M and the letter images $h(a)$ for $a \in \Sigma$. Thus $h(w)$ is computable by a left-to-right product in $O(|w|)$ time; this is the sense in which the fixed homomorphism is *efficiently computable* below.

For a finite semigroup S, an element $e \in \mathsf{S}$ is *idempotent* if $e^2 = e$; let $E(\mathsf{S})$ denote the set of all idempotents. We call S *locally trivial* if $ete = e$ for all $e \in E(\mathsf{S})$ and $t \in \mathsf{S}$. Every nonempty finite semigroup has an idempotent. Hence, if $\pi : \mathsf{T} \to \mathsf{S}$ is a surjective morphism of finite semigroups, each $e \in E(\mathsf{S})$ has an idempotent preimage f in the subsemigroup $\pi^{-1}(e)$, and S is locally trivial whenever T is, since then $e\pi(t)e = \pi(ftf) = e$. The *positive image semigroup* of h is $h(\Sigma^+)$; it need not contain the identity of M.

Definition 2.1 (fixed-h substitutability)**.** *A language $L \subseteq \Sigma^*$ is* $\sim_h$-substitutable *if for all $x, y \in \Sigma^+$, equality $h(x) = h(y)$ together with $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \varnothing$ implies $\mathcal{D}_L(x) = \mathcal{D}_L(y)$.*

The empty word is not used as an internal substitutable fragment. When $\lambda \in L$, the reconstruction handles it separately through the start symbol.

For a nonempty internal factor x, we call $h(x)$ its *yield type*. As above, $\mathcal{C}_h^{\mathrm{cf}}$ denotes the context-free $\sim_h$-substitutable languages, and we define $\mathcal{C}_h^{\mathrm{lin}} := \{L \in \mathcal{C}_h^{\mathrm{cf}} : L \text{ is generated by some linear CFG}\}$.

3 Fixed Recognizable Substitutability

For fixed h, put $\mathsf{RS}_h := \{L \subseteq \Sigma^* : L \text{ is } \sim_h\text{-substitutable}\}$. Then the fixed-$h$ slice is $\mathcal{C}_h^{\mathrm{cf}} = \mathsf{RS}_h \cap \mathrm{CFL}$, while $\mathcal{C}_h^{\mathrm{lin}}$ consists of its linearly generated members. For expressive comparisons only, let $\mathsf{RS} := \bigcup_h \mathsf{RS}_h$, where h ranges over finite-monoid homomorphisms on Σ^*. We write $\mathsf{RS}_h(\Gamma)$ and $\mathsf{RS}(\Gamma)$ when the alphabet matters; all inclusions below are alphabetwise. The learning results always concern one fixed h.

Proposition 3.1. *A language is regular iff it has the form $h^{-1}(\mathsf{Acc})$ for some homomorphism $h : \Sigma^* \to M$ into a finite monoid and some $\mathsf{Acc} \subseteq M$. For regular L, h may be taken to be its syntactic morphism, whose kernel is $\equiv_L$. For every fixed h and $\mathsf{Acc} \subseteq M$, the language $h^{-1}(\mathsf{Acc})$ is $\sim_h$-substitutable. Thus every regular language belongs to RS.*

Proof. The first two statements follow from the classical syntactic-monoid characterization [15, Chapter III, Propositions 10.1–10.2]. If $L = h^{-1}(\mathsf{Acc})$ and $h(x) = h(y)$, then $h(uxv) = h(uyv)$ for all u, v, so $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. □

By Proposition 3.1, RS and $\mathsf{RS} \cap \mathrm{CFL}$ contain every finite language and Σ^*; Gold's theorem makes both unions nonlearnable from positive data on nonempty alphabets. Yoshinaka [3, Section 4] notes the analogous issue for the union of fixed-window classes. We learn only fixed-h slices.

Typing refinement is monotone on the nonempty factors that enter Definition 2.1. If $\ker h'|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$, then $\mathsf{RS}_h \subseteq \mathsf{RS}_{h'}$, since h' requires fewer factor comparisons. Refinement may nevertheless enlarge the typed grammar used in the quantitative analysis. For two finite typings h, g, $\ker(h \times g)|_{\Sigma^+} = \ker h|_{\Sigma^+} \cap \ker g|_{\Sigma^+}$, hence $\mathsf{RS}_h \cup \mathsf{RS}_g \subseteq \mathsf{RS}_{h\times g}$; in particular, window information and another finite feature can be combined without making the typing target-specific.

Yoshinaka [3, Proposition 1] showed that fixed-window substitutable languages are not closed under intersection with regular languages or inverse images under homomorphisms that may erase. In contrast, they are closed under inverse images of nonerasing homomorphisms: for such a morphism φ, equality of fixed k-prefixes and ℓ-suffixes is preserved by φ. Indeed, $|\varphi(p)| \geq k$ for $p \in \Gamma^k$, so the first k letters of $\varphi(prq)$ depend only on p, and the suffix case is symmetric; short factors of equal window type are identical. Thus the fixed-window substitutability implication pulls back along φ. The following proposition shows that regular filtering and erasing inverse images can be accommodated by changing the finite typing.

Proposition 3.2 (closure under finite information)**.** *Let $h : \Sigma^* \to M$ and $g : \Sigma^* \to N$ be finite-monoid homomorphisms.*

(i) *If $L \in \mathsf{RS}_h$ and $J \in \mathsf{RS}_g$, then $L \cap J \in \mathsf{RS}_{h\times g}$.*

(ii) *If $R_{\mathrm{reg}} = g^{-1}(\mathsf{Acc}_g)$ is regular and $L \in \mathcal{C}_h^{\mathrm{cf}}$, then $L \cap R_{\mathrm{reg}} \in \mathcal{C}_{h\times g}^{\mathrm{cf}}$.*

(iii) *Let $\varphi : \Gamma^* \to \Sigma^*$ be any homomorphism, possibly erasing. Let $U_1 = \{1, \zeta\}$ be the two-element monoid with zero ζ, define $e_\varphi(w) = 1$ iff $\varphi(w) = \lambda$ and $e_\varphi(w) = \zeta$ otherwise, and put $\widehat{h} = (h \circ \varphi) \times e_\varphi$.[1] If $L \in \mathsf{RS}_h$, then $\varphi^{-1}(L) \in \mathsf{RS}_{\widehat{h}}$; if moreover $L \in \mathcal{C}_h^{\mathrm{cf}}$, then $\varphi^{-1}(L) \in \mathcal{C}_{\widehat{h}}^{\mathrm{cf}}$.*

Proof. For (i), suppose nonempty x, y have equal $(h\times g)$-type and share a context in $L \cap J$. They then have equal h-type and a common context in L, and equal g-type and a common context in J. Hence $\mathcal{D}_L(x) = \mathcal{D}_L(y)$ and $\mathcal{D}_J(x) = \mathcal{D}_J(y)$, so $\mathcal{D}_{L\cap J}(x) = \mathcal{D}_{L\cap J}(y)$. Item (ii) follows from (i), Proposition 3.1, and closure of CFLs under intersection with regular languages.

For (iii), e_φ is a homomorphism because $\varphi(uv) = \lambda$ iff both $\varphi(u)$ and $\varphi(v)$ are empty. Let equal-$\widehat{h}$-type nonempty x, y share a context in $\varphi^{-1}(L)$. If $e_\varphi(x) = e_\varphi(y) = 1$, then $\varphi(x) = \varphi(y) =$

[1] The flag e_φ cannot in general be omitted. For $L = a^+$ and trivial h, let $\varphi(a) = a$ and $\varphi(b) = \lambda$. In $\varphi^{-1}(L)$, the factors a, b have equal $(h \circ \varphi)$-types and share the context (a, λ), but a belongs to the inverse image whereas b does not.

λ, so the two factors have identical distributions in the inverse image. Otherwise both images are nonempty, have equal h-type, and the shared inverse-image context maps to a common context of $\varphi(x), \varphi(y)$ in L. Fixed-h substitutability in L gives equal distributions there, and pulling contexts back through φ gives $\mathcal{D}_{\varphi^{-1}(L)}(x) = \mathcal{D}_{\varphi^{-1}(L)}(y)$. The context-free statement uses closure of CFLs under inverse homomorphism. □

Typing adequacy. By Definition 2.1, $L \in \mathsf{RS}_h$ exactly when $\ker h$ identifies no two nonempty factors having a common context but different distributions; typing adequacy is thus a finite-index monoid-congruence separation condition, preserved by refinement and product typing at a possible quantitative cost (Proposition 6.3). The stronger requirement $\ker h|_{\Sigma^+} \subseteq \equiv_L$ is sufficient but forces L to be regular.

3.1 Classical special cases

Clark–Eyraud's original Definition 5 includes all fragments [2]; we follow Clark [19] in excluding λ. This distinction matters even for $L = a^+$: (a, λ) belongs to both $\mathcal{D}_L(\lambda)$ and $\mathcal{D}_L(a)$, whereas (λ, λ) belongs to $\mathcal{D}_L(a)$ but not to $\mathcal{D}_L(\lambda)$. Thus comparing λ with nonempty factors would exclude a^+. Call L *Clark–Eyraud substitutable* when $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ implies $\mathcal{D}_L(x) = \mathcal{D}_L(y)$ for all $x, y \in \Sigma^+$. This is exactly the trivial-monoid case of Definition 2.1, and hence also Yoshinaka's $(0, 0)$ case.

For $k, \ell \geq 0$, let $\mathrm{pref}_k(w)$ and $\mathrm{suf}_\ell(w)$ denote the prefix and suffix of lengths $\min\{k, |w|\}$ and $\min\{\ell, |w|\}$, respectively; in particular, $\mathrm{pref}_0(w) = \mathrm{suf}_0(w) = \lambda$. Renaming Yoshinaka's variables so that u_i, v_i denote outer contexts, L is (k, ℓ)-*substitutable* if, for all $p \in \Sigma^k$, $q \in \Sigma^\ell$, and $u_1, u_2, v_1, v_2, r, s \in \Sigma^*$ with $prq, psq \neq \lambda$,

$$u_1 prq v_1 \in L, \quad u_1 psq v_1 \in L, \quad u_2 prq v_2 \in L \\ \implies \quad u_2 psq v_2 \in L.$$

Proposition 3.3. *For every fixed $k, \ell \geq 0$ there is a finite monoid homomorphism $h_{k,\ell}$ such that, for every $L \subseteq \Sigma^*$, L is (k, ℓ)-substitutable iff it is $\sim_{h_{k,\ell}}$-substitutable.*

Proof. Put $m_0 := \max\{1, k + \ell\}$, let $h_{k,\ell}(w) := w$ for $|w| < m_0$, and otherwise let $h_{k,\ell}(w) := \langle \mathrm{pref}_k(w), \mathrm{suf}_\ell(w) \rangle$, with a tag separating the two cases. Denote the finite image by $M_{k,\ell} := h_{k,\ell}(\Sigma^*)$. On $M_{k,\ell}$ define, for x, y with $|x|, |y| < m_0$,

$$x \cdot y := h_{k,\ell}(xy), \qquad x \cdot \langle p, q \rangle := \langle \mathrm{pref}_k(xp), q \rangle, \\ \langle p, q \rangle \cdot y := \langle p, \mathrm{suf}_\ell(qy) \rangle, \quad \langle p, q \rangle \cdot \langle p', q' \rangle := \langle p, q' \rangle.$$

Then $h_{k,\ell}(xy) = h_{k,\ell}(x) \cdot h_{k,\ell}(y)$; surjectivity gives associativity and $h_{k,\ell}(\lambda)$ is the identity.

For nonempty x, y, equal types mean either $x = y$ or $x = prq$, $y = psq$ for a common length-k prefix p and length-ℓ suffix q (including $k = \ell = 0$). If L is (k, ℓ)-substitutable, a shared context plus any context of one factor instantiates Yoshinaka's four-word condition, and symmetry gives equal distributions. Conversely, in any Yoshinaka premise the factors $x := prq$ and $y := psq$ are nonempty and lie in the long-word part of the coding, hence have equal $h_{k,\ell}$-type; fixed-$h_{k,\ell}$ substitutability transfers the third context and yields the fourth word. □

Yoshinaka [8, Example 4] already represented (k, ℓ)-substitutability by the sorted signature $\Sigma_{k,\ell}$. Proposition 3.3 makes the inducing finite-monoid morphism explicit; no priority for the window-signature encoding is claimed.

Fixed-window counterexample criterion. If x, y have length at least $\max\{1, k+\ell\}$, the same length-k prefix and length-ℓ suffix, a common context, and unequal distributions, then L is not (k,ℓ)-substitutable: the first three conditions give equal $h_{k,\ell}$-types and a shared context, while the last contradicts $\sim_{h_{k,\ell}}$-substitutability.

Thus the fixed (k,ℓ) class is exactly $\mathsf{RS}_{h_{k,\ell}}$. Section 7 gives its polynomial thick-data bound. For $k = \ell = 0$, $h_{0,0}$ merely separates λ from nonempty words and induces the same substitutability condition as the one-element monoid. For an alphabet Γ, set

$$\mathsf{KL}(\Gamma) := \bigcup_{k,\ell \geq 0} \{L \subseteq \Gamma^* : L \text{ is } (k,\ell)\text{-substitutable}\},$$

and abbreviate $\mathsf{KL} := \mathsf{KL}(\Sigma)$.

3.2 Main theorem

Theorem 3.4 (fixed-h learning theorem)**.** *Fix an efficiently computable homomorphism $h : \Sigma^* \to M$ into a finite monoid. There are a set-driven reconstruction operator Rec_h and a conservative sequential learner Learn_h such that:*

(i) *Rec_h is sample-consistent and $\mathsf{Rec}_h(K)$ is constructible in time polynomial in $\|K\|$;*

(ii) *every $L \in \mathcal{C}_h^{\mathrm{cf}}$ has a finite characteristic sample for Rec_h;*

(iii) *Learn_h identifies every nonempty $L \in \mathcal{C}_h^{\mathrm{cf}}$ in Gold's sense, each update taking time polynomial in the data seen so far;*

(iv) *if the positive image semigroup $h(\Sigma^+)$ is a homomorphic image of a subsemigroup of the direct product of a finite locally trivial semigroup and a finite group, then for every nonempty target the sample in (ii) can be chosen polynomial in $|G_*|$ and $\tau_{G_*} + 1$ for any reduced CFG G_* representing that target; this includes every fixed-window and every finite-group typing;*

(v) *for arbitrary fixed h and $L \in \mathcal{C}_h^{\mathrm{lin}}$, the sample in (ii) can be chosen polynomial in $|G_*|$ for any linear CFG G_* representing L.*

The polynomials in (iv)–(v) may depend on the fixed h and Σ, but not on the chosen target grammar G_*. The quantitative mechanism behind items (iv)–(v), together with its limitation for arbitrary fixed typings, is developed in Sections 6–8.

This is consistent with Gold's theorem: on a nonempty alphabet choose distinct $x, y \in \Sigma^+$ with $h(x) = h(y)$, and choose $z \in \Sigma^*$ with $|z| > \max\{|x|, |y|\}$. The finite language $\{x, y, zx\}$ is not $\sim_h$-substitutable: x, y share the empty context but only x admits (z, λ).

Algebraic position of fixed windows. The following positive-image/kernel criterion is classical [20, Propositions XI.4.17 and XIV.1.20]; the generalized definite language characterization is likewise known [21, Proposition 5.18]. Its learning-specific application is Corollary 7.5.

Proposition 3.5 (locally trivial typings and fixed windows)**.** *Let $h : \Sigma^* \to M$ be a finite-monoid homomorphism on a nonempty alphabet, and put $n := |h(\Sigma^+)| + 1$. The following are equivalent:*

(i) *$h(\Sigma^+)$ is locally trivial;*

(ii) *there are $k, \ell \geq 0$ such that $\ker h_{k,\ell}|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$.*

If these conditions hold, $(k,\ell) = (n,n)$ works. Moreover every positive image semigroup $h_{k,\ell}(\Sigma^+)$ is locally trivial, and therefore

$$\mathsf{KL}(\Sigma) = \bigcup_{\substack{h:\Sigma^* \to M \\ h(\Sigma^+) \text{ locally trivial}}} \mathsf{RS}_h(\Sigma).$$

The same equality holds after intersecting both sides with CFL.

Proof. Put $\mathsf{S} = h(\Sigma^+)$ and $n = |\mathsf{S}| + 1$. First, every $h_{k,\ell}(\Sigma^+)$ is locally trivial: no short-word type is idempotent, since squaring either changes the represented short word or enters the long-word component, and by the multiplication in Proposition 3.3 every long-word type satisfies $\langle p, q\rangle t \langle p, q\rangle = \langle p, q\rangle$ for every t in the positive image.

(i)⇒(ii). If S is locally trivial, the classical identities $\mathsf{S}^n = \mathsf{S}E(\mathsf{S})\mathsf{S}$ and $esf = ef$ for $e, f \in E(\mathsf{S})$ and $s \in \mathsf{S}$ hold [20, Corollary II.6.35 and Proposition XI.4.17]. For $p, q \in \Sigma^n$, write $h(p) = aeb$ and $h(q) = cfd$, where e, f are idempotent. Then, for every $r \in \Sigma^*$,

$$h(prq) = a\,e\,(b\,h(r)\,c)\,f\,d = aefd = h(pq).$$

Thus the length-n prefix and suffix determine $h(w)$ whenever $|w| \geq 2n$. Since $h_{n,n}$ distinguishes shorter words individually, (ii) follows with $(k, \ell) = (n, n)$.

(ii)⇒(i). The kernel inclusion makes $h_{k,\ell}(w) \mapsto h(w)$ a well-defined surjective morphism from the locally trivial semigroup $h_{k,\ell}(\Sigma^+)$ onto S, so S is locally trivial (Section 2.2). The union equalities follow from these facts and kernel-refinement monotonicity, also after restriction to CFL. □

4 Reconstruction Operator and Sequential Learner

4.1 Finite-sample reconstruction

For a finite sample $K \subseteq \Sigma^*$, let

$$\widehat{V}(K) := \{[\![x]\!] : x \in \Sigma^+,\ \exists u, v \in \Sigma^* \text{ with } uxv \in K\}.$$

Thus there is one hypothesis nonterminal for each distinct nonempty factor observed in the sample; the double brackets are part of the nonterminal name and do not denote a quotient class or literal terminal symbols. For observed nonempty factors x, y, write

$$x \approx_{K,h} y \quad\Longleftrightarrow\quad h(x) = h(y) \ \text{ and } \ \mathcal{D}_K(x) \cap \mathcal{D}_K(y) \neq \emptyset.$$

The grammar $\mathsf{Rec}_h(K)$ has nonterminal set $\widehat{V}(K) \cup \{\widehat{S}\}$, start symbol $\widehat{S}$, and the following *set* of productions whenever every displayed bracketed nonterminal belongs to $\widehat{V}(K)$:

$$\begin{array}{lll} \text{(B)} & [\![xy]\!] \to [\![x]\!][\![y]\!], & x, y \in \Sigma^+, \\ \text{(U)} & [\![x]\!] \to [\![y]\!], & x \approx_{K,h} y, \\ \text{(L)} & [\![a]\!] \to a, & a \in \Sigma, \\ \text{(S)} & \widehat{S} \to [\![w]\!], & w \in K \cap \Sigma^+. \end{array}$$

If $\lambda \in K$, Rule (S) also contains $\widehat{S} \to \lambda$.

For finite samples $K \subseteq K'$, every nonterminal and production of $\mathsf{Rec}_h(K)$ also occurs in $\mathsf{Rec}_h(K')$. Consequently, $L(\mathsf{Rec}_h(K)) \subseteq L(\mathsf{Rec}_h(K'))$.

The construction follows the substring-indexed grammar scheme of Yoshinaka [3, Section 4, Algorithm 1], also recalled by Clark [19, Section 4, Algorithm 1]. For $h = h_{k,\ell}$, the nontrivial instances of Rule (U) coincide with Yoshinaka's unary rules: distinct observed nonempty factors are connected when they share a sample context and have equal $h_{k,\ell}$-type; equivalently, both factors have length at least $\max\{1, k + \ell\}$ and have the same length-k prefix and length-ℓ suffix. The constructions agree up to redundant identity rules, the representation of initial productions by a separate start symbol, and the treatment of λ. For general fixed h, the prefix–suffix test is replaced by equality of h-types.

Lemma 4.1 (sample consistency)**.** *For every finite sample K, $K \subseteq L(\mathsf{Rec}_h(K))$.*

Proof. If $w = a_1 \cdots a_m \in K \cap \Sigma^+$, Rule (S) enters $[\![w]\!]$. Repeated use of Rule (B) along any binary factorization of w reaches the one-letter factors, all of which are observed because they occur inside the same sample word. Rule (L) then emits $a_1 \cdots a_m$. If $\lambda \in K$, the start rule emits it directly. □

Theorem 4.2 (soundness)**.** *If $K \subseteq L$ and L is $\sim_h$-substitutable, then $L(\mathsf{Rec}_h(K)) \subseteq L$.*

Proof. Induct on the height of a derivation tree rooted at $[\![x]\!]$ to prove

$$[\![x]\!] \Rightarrow^*_{\mathsf{Rec}_h(K)} w \quad \Longrightarrow \quad x \equiv_L w \ \text{ and } \ h(w) = h(x). \tag{1}$$

Every factor nonterminal is indexed by a nonempty observed word, so the fragments compared by Rule (U) satisfy the nonempty requirement of Definition 2.1. Moreover, Rules (B)–(L) cannot derive λ from a factor nonterminal. Rule (L) is immediate. For Rule (U), suppose $[\![x]\!] \to [\![y]\!]$. By definition there is a sample context (u, v) with $uxv, uyv \in K \subseteq L$ and $h(x) = h(y)$. Fixed-h substitutability gives $x \equiv_L y$; the induction hypothesis gives $y \equiv_L w$ and $h(w) = h(y)$, proving (1).

For Rule (B), write the derived word as $w_1 w_2$ from $[\![xy]\!] \to [\![x]\!][\![y]\!]$. The induction hypotheses give $x \equiv_L w_1$, $y \equiv_L w_2$, $h(w_1) = h(x)$, and $h(w_2) = h(y)$. Because $\equiv_L$ is a monoid congruence, $xy \equiv_L w_1 w_2$, and the homomorphism law gives $h(w_1 w_2) = h(xy)$.

Finally, if a start rule $\widehat{S} \to [\![x]\!]$ is used, then $x \in K \subseteq L$, so $(\lambda, \lambda) \in \mathcal{D}_L(x)$. The invariant gives $\mathcal{D}_L(w) = \mathcal{D}_L(x)$ and hence $w \in L$. The direct λ rule is present only when $\lambda \in K \subseteq L$. □

4.2 The sequential learner

Recomputing Rec_h after every datum need not stabilize syntactically: for $L_c = \{a^n c b^n : n \geq 0\}$, longer examples introduce new factor nonterminals such as $[\![a^n c b^n]\!]$. We therefore use the conservative wrapper of Clark–Eyraud's Substitution Graph Learner (SGL, Algorithm 2) and Yoshinaka's (k, ℓ)-Substitutable Grammar Learner ((k, ℓ)-SGL, Algorithm 1) [2, 3]; see also Eyraud–Heinz–Yoshinaka [17, Appendix] for the distinction between set-driven reconstruction and incremental learning. Starting from the fixed empty grammar H_0, after reading w_n put

$$H_n := \begin{cases} H_{n-1}, & w_n \in L(H_{n-1}), \\ \mathsf{Rec}_h(K_n), & w_n \notin L(H_{n-1}), \end{cases} \qquad \mathsf{Learn}_h(\mathcal{T}[n]) := H_n.$$

Every rebuild uses the whole accumulated sample K_n. Although Rec_h is set-driven, the grammar on which Learn_h stabilizes may depend on the presentation order; only eventual syntactic stabilization to a grammar for the target is claimed (Corollary 5.5). This order dependence is genuine: for any infinite target and a convergent text $\mathcal{T}$, choose $w \in L$ longer than all examples at the last rebuild. The limiting grammar for $\mathcal{T}$ omits $[\![w]\!]$, but prepending w to $\mathcal{T}$ puts $[\![w]\!]$ in every rebuilt grammar, so the two limiting grammars differ. See also Clark [19, Section 4, Example 1] for this phenomenon with $L_c = \{a^n c b^n : n \geq 0\}$.

5 Finite Witnesses, Exact Reconstruction, and Complexity

5.1 Target refinement and finite witnesses

For the completeness proof only, we refine a target grammar by yield type and show that the hypothesis can mimic finitely witnessed typed derivations. We use the following normal form as a proof device.

Definition 5.1 (start-separated binary normal form)**.** *A CFG $G = (V, \Sigma, P, S_0)$ is in* start-separated binary normal form *(SSBNF) if S_0 never occurs on a right-hand side, every start rule is $S_0 \to A$ or $S_0 \to \lambda$, and every non-start rule is either $A \to a$ or $A \to BC$.*

The start separation isolates whole-sentence entry from internal nonterminals and allows λ to be handled at the start symbol.

Every nonempty context-free language has an equivalent reduced SSBNF grammar by the standard start-separation, useless-symbol elimination, λ/unit elimination away from the start, and binarization procedures. The refinement below is therefore defined for nonempty context-free languages; the empty target is represented separately by H_0 and $\mathsf{Rec}_h(\emptyset)$.

Let $G = (V, \Sigma, P, S_0)$ be a reduced SSBNF grammar for a nonempty language L. Form the following yield-typed grammar with start symbol $\widetilde{S}$ and non-start symbols A_μ with $A \in V \setminus \{S_0\}$ and $\mu \in M$. Here and below, juxtaposition of type indices denotes multiplication in M; thus $\mu\nu = \mu \cdot \nu$. The rules are

$$\begin{array}{ll} \widetilde{S} \to A_\mu & (S_0 \to A \in P,\ \mu \in M), \\ \widetilde{S} \to \lambda & (S_0 \to \lambda \in P), \\ A_{h(a)} \to a & (A \to a \in P), \\ A_{\mu\nu} \to B_\mu C_\nu & (A \to BC \in P,\ \mu, \nu \in M). \end{array}$$

The reduced part of this grammar is denoted $\widetilde{G} = (\widetilde{V}, \Sigma, \widetilde{P}, \widetilde{S})$ and called the *typed refinement*. Since $\widetilde{G}$ is obtained by trimming the yield-typed grammar, every remaining symbol is reachable and productive.

Fix an order on Σ and use the induced *shortlex* order: shorter strings come first, with lexicographic order breaking equal-length ties. For every non-start symbol X of $\widetilde{G}$, let $\omega(X)$ be its shortlex-minimal terminal yield, called the *canonical yield* of X. This is well-defined because $\widetilde{G}$ is reduced.

The yield-type split is used in the present completeness proof: the canonical yield of a typed parent and the concatenated canonical yields of its typed children must have equal h-types to license unary Rule (U) (Theorem 5.3).

The role of this split in the present completeness argument can already be seen before typing. Let $h(w) = |w| \bmod 2$ and let G be the SSBNF grammar with start symbol S_0 and rules $S_0 \to A$, $A \to a \mid BC$, $B \to a$, and $C \to a$. The shortest yield of A is a, of odd type, while the concatenation of shortest yields of B and C is aa, of even type. Without separating A by yield type, the corresponding unary-rule step is therefore not licensed. Splitting by yield type makes the required equality an invariant. By contrast, no types are attached to the surrounding left or right contexts, because the reconstruction operator never tests such types.

Proposition 5.2 (typed lifting and yield invariant)**.** *The typed refinement satisfies $L(\widetilde{G}) = L(G)$. For every retained non-start symbol A_μ,*

$$L_{\widetilde{G}}(A_\mu) = L_G(A) \cap h^{-1}(\{\mu\}).$$

In particular, $A_\mu \Rightarrow^*_{\widetilde{G}} w$ implies $h(w) = \mu$.

Proof. The terminal and binary rules above are exactly the bottom-up propagation laws for yield types. Hence, by induction on typed derivations, $A_\mu \Rightarrow^*_{\widetilde{G}} w$ implies $h(w) = \mu$, and erasing type annotations maps typed derivations to derivations of G; this gives $L(\widetilde{G}) \subseteq L(G)$ and the inclusion $\subseteq$ in the display.

Conversely, annotating each non-start node of a derivation tree of G by the h-type of its yield gives a derivation tree of the untrimmed yield-typed grammar. For a successful tree all its symbols survive trimming, so $L(G) \subseteq L(\widetilde{G})$. For retained A_μ and $w \in L_G(A)$ with $h(w) = \mu$, graft the annotated tree of $A \Rightarrow^*_G w$, whose root has type μ, at an occurrence of A_μ in a successful typed tree; the result is successful, so its symbols and rules survive trimming and $A_\mu \Rightarrow^*_{\widetilde{G}} w$. □

For every non-start symbol X of $\widetilde{G}$, choose $\chi(X) = (u_X, v_X)$ among the terminal reaching contexts of X so as to minimize $|u_X| + |v_X|$, breaking ties by the shortlex orders of u_X and

then v_X; call $\chi(X)$ the *canonical context* of X. Such a terminal reaching context exists because $\widetilde{G}$ is reduced: reachability places X in a reachable sentential form, and productivity of the surrounding nonterminals lets all symbols to its left and right be expanded to terminal strings. Proposition 5.2 gives $h(\omega(A_\mu)) = \mu$ for every typed non-start symbol.

Define the finite witness set

$$\begin{aligned}\mathsf{Wit}(\widetilde{G}) := {} & \{u_X\omega(X)v_X : X \in \widetilde{V} \setminus \{\widetilde{S}\}\} \\ & \cup \{u_X a v_X : X \to a \in \widetilde{P}\} \\ & \cup \{u_X\omega(Y)\omega(Z)v_X : X \to YZ \in \widetilde{P}\} \\ & \cup \big(\{\lambda\} \text{ if } \lambda \in L\big).\end{aligned} \tag{2}$$

Every displayed word belongs to L, and the set is finite. In particular, for a binary rule $X \to YZ$ the word in the third family is justified by $\widetilde{S} \Rightarrow^* u_X X v_X \Rightarrow u_X Y Z v_X \Rightarrow^* u_X\omega(Y)\omega(Z)v_X$; the first and second families are justified analogously. We call the elements of $\mathsf{Wit}(\widetilde{G})$ the *canonical positive witnesses*. The first family records one canonical occurrence of each typed symbol, and the second and third families record the terminal and binary productions, respectively.

5.2 Exact reconstruction and Gold convergence

Theorem 5.3 (completeness from the finite witnesses)**.** *Let G be any reduced SSBNF grammar for a nonempty language L, and let $\widetilde{G}$ be its typed refinement with respect to the fixed homomorphism h. If a finite set K satisfies $\mathsf{Wit}(\widetilde{G}) \subseteq K$, then $L \subseteq L(\mathsf{Rec}_h(K))$.*

Neither substitutability nor $K \subseteq L$ is needed for this combinatorial completeness statement.

Proof. For each non-start X write $\widehat{X} := [\![\omega(X)]\!]$. The corresponding word in the first family of (2) ensures that $\widehat{X}$ exists. We prove by induction on a typed derivation $X \Rightarrow^* w$ that $\widehat{X} \Rightarrow^*_{\mathsf{Rec}_h(K)} w$.

If the first rule is $X \to a$, the first and second witness families contain $u_X\omega(X)v_X$ and $u_X a v_X$, respectively. Hence $\omega(X) \approx_{K,h} a$ because $h(\omega(X)) = h(a)$ by Proposition 5.2. Rule (U) moves $\widehat{X}$ to $[\![a]\!]$ when necessary, and Rule (L) yields a.

Suppose the first rule is $X = A_{\mu\nu} \to B_\mu C_\nu$; put $Y := B_\mu$ and $Z := C_\nu$. The first and third witness families contain $u_X\omega(X)v_X$ and $u_X\omega(Y)\omega(Z)v_X$. Since $h(\omega(X)) = \mu\nu = h(\omega(Y))h(\omega(Z)) = h(\omega(Y)\omega(Z))$, Rule (U) moves $\widehat{X}$ to $[\![\omega(Y)\omega(Z)]\!]$ when necessary. The third witness also makes $\omega(Y)\omega(Z)$ an observed factor, so Rule (B) gives $[\![\omega(Y)\omega(Z)]\!] \to [\![\omega(Y)]\!][\![\omega(Z)]\!]$. The induction hypotheses finish the two subderivations.

Finally, every nonempty $w \in L$ has a lifted start derivation $\widetilde{S} \to X \Rightarrow^* w$. The start rule itself shows that (λ, λ) is a terminal reaching context of X; its total length is 0, which is minimal, so the canonical choice is $\chi(X) = (\lambda, \lambda)$. The first witness family therefore gives $\omega(X) \in K$, and Rule (S) starts the preceding derivation. The case $w = \lambda$ is immediate from (2) and Rule (S). □

Theorem 5.4 (exact finite-sample reconstruction)**.** *Let $L \in \mathcal{C}_h^{\mathrm{cf}}$ be nonempty, let G be a reduced SSBNF grammar for L, and let $\widetilde{G}$ be its typed refinement. If $\mathsf{Wit}(\widetilde{G}) \subseteq K \subseteq L$, then $L(\mathsf{Rec}_h(K)) = L$.*

Proof. Combine Theorems 4.2 and 5.3. □

Thus $\mathsf{Wit}(\widetilde{G})$ is characteristic for every nonempty target, including $\{\lambda\}$. For $L = \emptyset$, the empty sample gives $L(\mathsf{Rec}_h(\emptyset)) = \emptyset$. This proves Theorem 3.4(ii).

Corollary 5.5 (Gold identification)**.** *For a fixed homomorphism $h : \Sigma^* \to M$ into a finite monoid M, the learner Learn_h identifies every nonempty language in $\mathcal{C}_h^{\mathrm{cf}}$ from positive data. More precisely, for any reduced SSBNF target grammar G and its typed refinement $\widetilde{G}$, once $\mathsf{Wit}(\widetilde{G}) \subseteq K_{n_0}$, among stages $n \geq n_0$ at most one hypothesis change can occur.*

Proof. Let $\mathcal{T}$ be a text for a nonempty L, and choose n_0 with $\mathsf{Wit}(\widetilde{G}) \subseteq K_{n_0}$. Since $w_1 \notin L(H_0) = \emptyset$, the learner rebuilds at stage 1, so for $n \geq 1$ the hypothesis H_n is $\mathsf{Rec}_h(K_m)$ for the most recent rebuild stage $m \leq n$. Sample consistency of that rebuild and the rebuild rule for the later data give $K_n \subseteq L(H_n)$, and soundness gives $L(H_n) \subseteq L$, as $K_m \subseteq L$. If a rebuild occurs at some $n \geq n_0$, Theorem 5.4 gives $L(H_n) = L$, and no later datum triggers a change. Otherwise $H_n = H_{n_0}$ for all $n \geq n_0$, so every word of $\mathcal{T}$ lies in some $K_n \subseteq L(H_n) = L(H_{n_0})$ with $n \geq n_0$; since every word of L occurs in $\mathcal{T}$, $L(H_{n_0}) = L$. In both cases at most one change occurs after stage n_0. □

A missed-positive rebuild can be delayed after a characteristic sample appears; compare Eyraud–Heinz–Yoshinaka [17, Appendix].

5.3 Complexity of the reconstruction step

Theorem 5.6 (polynomial-time reconstruction)**.** *For fixed h, $\mathsf{Rec}_h(K)$ is constructible in time polynomial in $\|K\|$.*

Proof. Put $n_K := \|K\|$ and let $\mathcal{O}_K := \{(u, x, v) \in \Sigma^* \times \Sigma^+ \times \Sigma^* : uxv \in K\}$ be the set of observed nonempty factor occurrences. Since $|\mathcal{O}_K| = O(n_K^2)$, there are at most $O(n_K^2)$ factor nonterminals. Extending factors one symbol at a time from each starting position of each sample word computes and caches all their h-values using $O(n_K^2)$ multiplications in the fixed monoid.

Rule (B) has at most $O(n_K^3)$ candidates, obtained by choosing an observed factor occurrence and a split point. For Rule (U), for each $(u, x, v) \in \mathcal{O}_K$ and each $w \in K$, check whether there is a nonempty word y with $w = uyv$. Such a y, if it exists, is unique; if $h(x) = h(y)$, emit the rule $[\![x]\!] \to [\![y]\!]$. This enumerates all Rule (U) candidates, at most $|\mathcal{O}_K||K| = O(n_K^3)$ of them.

Lexical and start rules are lower-order. Checking, encoding and inserting a candidate take $O(n_K)$ time, and a trie of the explicit rule encodings removes duplicates, so the output is a rule set independent of the enumeration order. Thus $\mathsf{Rec}_h(K)$ is constructible in $O(n_K^4)$ time. □

Each update of Learn_h is also polynomial in the data seen so far, since CFG membership is polynomial and hypotheses and rebuilds are polynomial by Theorem 5.6. Together with sample consistency (Section 4), Theorem 5.4 and Corollary 5.5, this proves Theorem 3.4(i)–(iii); items (iv)–(v) are proved in Sections 7–8. Grammar size alone cannot bound witness lengths: $S_0 \to D_n$, $D_0 \to a$, $D_i \to D_{i-1}D_{i-1}$ has size $O(n)$ but generates $\{a^{2^n}\}$, substitutable for every h [2, Example 2], [17, Example 2.3].

6 Typed Thickness for General Fixed Typings

For an arbitrary fixed h, the finite-witness construction admits a direct quantitative bound once the cost of yield typing is measured explicitly. Let $G = (V, \Sigma, P, S_0)$ be a reduced SSBNF grammar and let $\widetilde{G}$ be its typed refinement. Put $N := |V \setminus \{S_0\}|$ and $N_t := |\widetilde{V} \setminus \{\widetilde{S}\}|$ and define the *typed thickness* of this presentation by

$$\tau_h^{\mathrm{typ}}(G) := \max\Big(\{\, |\omega(X)| : X \in \widetilde{V} \setminus \{\widetilde{S}\}\,\} \cup \{1\}\Big).$$

Thus $\tau_h^{\mathrm{typ}}(G)$ measures shortest terminal yields after the full yield-type split and trimming. It is a presentation-dependent parameter, not an additional assumption on the learner.

Lemma 6.1 (typed-thickness witness bound)**.** *Let G and $\widetilde{G}$ be as above and write $\tau := \tau_h^{\mathrm{typ}}(G)$. Every non-start typed symbol X has $|u_X|+|v_X| \leq (N_t-1)\tau$, and every word $w \in \mathsf{Wit}(\widetilde{G})$ satisfies $|w| \leq (N_t+1)\tau$. If P_{ter} and P_{bin} are respectively the terminal and binary non-start productions of G, then*

$$\|\mathsf{Wit}(\widetilde{G})\| \leq \big(N|M| + |P_{\mathrm{ter}}| + |P_{\mathrm{bin}}||M|^2 + 1\big)\big((N|M|+1)\tau + 1\big).$$

In particular, for fixed h this bound is polynomial in $|G|$ and $\tau_h^{\mathrm{typ}}(G)$.

Proof. Form the directed dependency graph whose vertices are the non-start symbols of $\widetilde{G}$ and whose edges join a parent to each child occurring in a binary production. Since $\widetilde{G}$ is reduced, every non-start symbol X is reachable. Hence some start child A has a path to X in this graph. Choose a shortest such path. It repeats no typed symbol, so it has at most $N_t - 1$ edges.

For each edge of the path, choose a witnessing binary production and expand the child not lying on the path to its canonical yield. Starting from $\widetilde{S} \to A$, these expansions produce a terminal reaching context of X. Each off-path canonical yield has length at most τ, so the total context length is at most $(N_t - 1)\tau$. The canonical context (u_X, v_X) is no longer than this one.

A word in the first family adds one canonical yield to the canonical context; the second family adds one terminal, and the third adds two canonical yields. Since $\tau \geq 1$, every word in (2) therefore has length at most $(N_t + 1)\tau$. Finally, $N_t \leq N|M|$; each source terminal production has at most one typed terminal copy, and each source binary production has at most $|M|^2$ typed copies. Thus the displayed first factor bounds the number of witnesses, including the possible witness λ, while the second factor bounds $|w| + 1$ for each witness. Multiplying the two bounds proves the claim. □

Corollary 6.2 (characteristic data from typed thickness)**.** *Fix h. If G is a reduced SSBNF grammar for a nonempty $L(G) \in \mathcal{C}_h^{\mathrm{cf}}$, then $\mathsf{Wit}(\widetilde{G})$ is a characteristic sample for Rec_h whose encoded size is polynomial in $|G|$ and $\tau_h^{\mathrm{typ}}(G)$.*

Proof. Lemma 6.1 gives the quantitative bound, and Theorem 5.4 gives the characteristic-sample property. □

The dependence on typed thickness in Corollary 6.2 cannot in general be replaced by ordinary thickness, even for the reconstruction operator itself under a single fixed typing.

A bracketed doubling family. Let $\Gamma_{\mathrm{br}} = \{\mathsf{a}, \lfloor, \rfloor, \lceil, \rceil\}$. Recall the two-element monoid $U_1 = \{1, \zeta\}$ from Section 3. Define $h_{\mathrm{U}} : \Gamma_{\mathrm{br}}^* \to U_1$ by

$$h_{\mathrm{U}}(\mathsf{a}) = h_{\mathrm{U}}(\lfloor) = h_{\mathrm{U}}(\rfloor) = 1, \qquad h_{\mathrm{U}}(\lceil) = h_{\mathrm{U}}(\rceil) = \zeta,$$

and let h_0 be the trivial typing. Thus $h_{\mathrm{U}}(w) = 1$ if and only if w contains neither $\lceil$ nor $\rceil$.

For $n \geq 1$, define the reduced CFG G_n by

$$S_0 \to E_n, \qquad E_0 \to \mathsf{a}, \qquad E_i \to \lfloor E_{i-1}E_{i-1}\rfloor \mid \lceil E_{i-1}\rceil \quad (1 \leq i \leq n),$$

and put $L_n = L(G_n)$. Then $|G_n| = O(n)$ and $\tau_{G_n} = 2n+1$. Write $w_0^* = \mathsf{a}$ and $w_i^* = \lfloor w_{i-1}^* w_{i-1}^* \rfloor$. The only word of L_n without ceiling brackets is w_n^*, of length $|w_n^*| = 3 \cdot 2^n - 2$: without unary ceiling-bracket nodes the derivation is the unique complete binary tree of height n.

Proposition 6.3 (refinement can force exponential characteristic data)**.** *For every $n \geq 1$ the following hold.*

(i) *L_n is Clark–Eyraud substitutable, and hence $L_n \in \mathcal{C}_{h_0}^{\mathrm{cf}} \cap \mathcal{C}_{h_{\mathrm{U}}}^{\mathrm{cf}}$.*

(ii) *L_n has a characteristic sample for Rec_{h_0} of encoded size polynomial in n.*

(iii) *Every finite $K \subseteq L_n$ such that $w_n^* \in L(\mathsf{Rec}_{h_{\mathrm{U}}}(K))$ contains w_n^*. Consequently, every characteristic sample $\mathcal{F}$ of L_n for $\mathsf{Rec}_{h_{\mathrm{U}}}$ satisfies $\|\mathcal{F}\| \geq 3 \cdot 2^n - 1$.*

(iv) *Put $L_n^- := L_n \setminus \{w_n^*\}$. Then $L_n^- \in \mathcal{C}_{h_{\mathrm{U}}}^{\mathrm{cf}}$. For any set-driven reconstruction operator Op with characteristic samples for both L_n and L_n^-, every characteristic sample $\mathcal{F}$ of L_n for Op contains w_n^* and hence satisfies $\|\mathcal{F}\| \geq 3 \cdot 2^n - 1$. In particular, this holds for every set-driven operator having characteristic samples for the entire fixed-h_{U} class.*

In particular, for the fixed typing h_{U} no bound on characteristic-data size for $\mathsf{Rec}_{h_{\mathrm{U}}}$ polynomial in source grammar size and ordinary thickness holds over all reduced CFG presentations of targets in $\mathcal{C}_{h_{\mathrm{U}}}^{\mathrm{cf}}$.

Proof. The substitutability assertion (i) is proved in Appendix C. For (ii), Proposition 7.3 gives a reduced SSBNF grammar G'_n of polynomial size and ordinary thickness. Since h_0 is trivial, $\tau^{\mathrm{typ}}_{h_0}(G'_n) \le \max\{1, \tau_{G'_n}\}$, so Corollary 6.2 applies.

An explicit sample is obtained by setting $u_0 = \mathtt{a}$, $u_i = \lceil u_{i-1} \rceil$, and

$$\mathcal{F}_n = \{u_n\} \cup \{\lceil^{n-i} \lfloor u_{i-1} u_{i-1} \rfloor \rceil^{n-i} : 1 \le i \le n\}.$$

For each i, the two sample words provide a common context for u_i and $\lfloor u_{i-1} u_{i-1} \rfloor$; Rules (U) and (B) therefore recover both E_i alternatives, so $L_n \subseteq L(\mathsf{Rec}_{h_0}(\mathcal{F}_n))$. For any $\mathcal{F}_n \subseteq K \subseteq L_n$, monotonicity applied to $\mathcal{F}_n \subseteq K$ gives $L_n \subseteq L(\mathsf{Rec}_{h_0}(K))$, while soundness (Theorem 4.2) applied to $K \subseteq L_n$ gives $L(\mathsf{Rec}_{h_0}(K)) \subseteq L_n$. Thus $\mathcal{F}_n$ is characteristic, with $|\mathcal{F}_n| = n + 1$ and $\|\mathcal{F}_n\| = 3n^2 + 4n + 2$.

For (iii)–(iv), put $L_n^- = L_n \cap h_{\mathrm{U}}^{-1}(\{\zeta\})$. By (i) and Proposition 3.2(ii), $L_n^- \in \mathcal{C}^{\mathrm{cf}}_{h_{\mathrm{U}} \times h_{\mathrm{U}}}$. Since $\ker(h_{\mathrm{U}} \times h_{\mathrm{U}}) = \ker h_{\mathrm{U}}$, we obtain $L_n^- \in \mathcal{C}^{\mathrm{cf}}_{h_{\mathrm{U}}}$.

For (iii), if $K \subseteq L_n$ omits w_n^*, then $K \subseteq L_n^-$. Soundness (Theorem 4.2) gives $L(\mathsf{Rec}_{h_{\mathrm{U}}}(K)) \subseteq L_n^-$, so w_n^* cannot be generated. Hence every characteristic sample $\mathcal{F}$ contains w_n^* and satisfies $\|\mathcal{F}\| \ge |w_n^*| + 1 = 3 \cdot 2^n - 1$. Together with $|G_n|, \tau_{G_n} = O(n)$, this proves the final assertion.

For (iv), let $\mathcal{F}$ be characteristic for L_n under Op and suppose $w_n^* \notin \mathcal{F}$. Take a characteristic sample $\mathcal{F}'$ for L_n^-. Then $\mathcal{F} \cup \mathcal{F}' \subseteq L_n^- \subsetneq L_n$, so the same output $\mathsf{Op}(\mathcal{F} \cup \mathcal{F}')$ would have to generate both languages, a contradiction.

For $n \ge 2$, $L_n^- \notin \mathcal{C}^{\mathrm{cf}}_{h_0}$: u_{n-1} and w_{n-1}^* share the context $(\lfloor, u_{n-1} \rfloor)$ in L_n^-, but only the former admits $(\lfloor w_{n-1}^*, \rfloor)$. Thus (iv) does not conflict with the h_0 upper bound in (ii). □

The contradiction in part (iv) follows the classical distinguishing-pair principle using a word in the symmetric difference of two targets; compare [4, Theorem 15.1.4]. The fixed-h distinguishing pair L_n, L_n^- and its exponential witness length are specific to the present construction.

Corollary 6.4 (presentation-independent typed-thickness gap)**.** *Let $G = (V, \Gamma_{\mathrm{br}}, P, S_0)$ be any reduced SSBNF grammar for L_n. Then*

$$\boxed{\tau^{\mathrm{typ}}_{h_{\mathrm{U}}}(G) \ge 3 \cdot 2^n - 2.}$$

The bound holds for every such reduced SSBNF presentation, without a restriction on its size. On the other hand, the proof of Proposition 6.3(ii) already gives a reduced SSBNF presentation of L_n with both grammar size and ordinary thickness polynomial in n.

Proof. Choose a successful derivation of w_n^* in G; its first production is $S_0 \to A$ for a non-start symbol A. The typed refinement retains A_1, because $A \Rightarrow_G^* w_n^*$ and $h_{\mathrm{U}}(w_n^*) = 1$. By Proposition 5.2,

$$L_{\widetilde{G}}(A_1) = L_G(A) \cap h_{\mathrm{U}}^{-1}(\{1\}) = \{w_n^*\},$$

since every yield of a start child belongs to L_n and w_n^* is its unique type-1 word. Thus the canonical yield of the retained non-start symbol A_1 is exactly w_n^*, so $\tau^{\mathrm{typ}}_{h_{\mathrm{U}}}(G) \ge |w_n^*|$. The second claim was shown in the proof of Proposition 6.3(ii). □

7 Polynomial Thick Data Beyond Fixed Windows

Yoshinaka [3, Theorem 1] already established polynomial thick-data learnability for fixed (k, ℓ)-substitutable context-free languages using (k, ℓ)-Greibach normal form and characteristic-set bounds [3, Lemmas 2 and 7]. Here we instead bound the shortest yields of the typed refinement used by Rec_h. One argument covers both fixed windows and typings that additionally track finite-group values. The corresponding batch bounds establish identification in polynomial time and thick data (IPTtD), in the set-driven sense of [3] and [17, Definition 2.12]. They do not

assert the stronger incremental stabilization requirement of [17, Definition 2.5]. We claim no improvement in Yoshinaka's polynomial degree. For general fixed h, ordinary thickness need not give a polynomial characteristic-data bound for Rec_h (Proposition 6.3).

Fix $k, \ell \geq 0$ and a morphism $g : \Sigma^* \to \mathsf{G}$ into a finite group. Put $m := k + \ell$ and $q_{\mathrm{grp}} := |\mathsf{G}|$.

Lemma 7.1 (window–group typed-yield bound)**.** *Let $G = (V, \Sigma, P, S_0)$ be a reduced SSBNF grammar and $N := |V \setminus \{S_0\}| \geq 1$. If a finite typing h satisfies $\ker(h_{k,\ell} \times g)|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$, then*

$$\tau_h^{\mathrm{typ}}(G) \leq m + \big[1 + 2(m + q_{\mathrm{grp}} - 1)(m + 1)N q_{\mathrm{grp}}\big]\tau_G.$$

Proof. Inverses in G compare group values before and after subtree replacement; the pigeonhole principle then gives a shortening interval that preserves the group value. See Appendix A for details. □

Theorem 7.2 (polynomial thick data under window–group refinement)**.** *Fix h, Σ and suppose that $\ker(h_{k,\ell} \times g)|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$ for some fixed k, ℓ and finite-group morphism g. Every nonempty $L \in \mathcal{C}_h^{\mathrm{cf}}$ represented by a reduced SSBNF grammar G with a non-start symbol has a characteristic sample $\mathsf{Wit}(\widetilde{G})$ for Rec_h whose encoded size is polynomial in $|G|$ and τ_G.*

Proof. Apply Lemma 7.1 and Corollary 6.2. The singleton target $\{\lambda\}$ is handled separately. □

Theorem 7.2 includes fixed windows by taking G trivial. The following normalization transfers the bound to arbitrary reduced CFG presentations.

Proposition 7.3 (polynomial thickness-preserving SSBNF normalization)**.** *There are fixed polynomials q_{size} and q_{thick} such that every reduced CFG G_* generating a nonempty language can be converted in polynomial time into an equivalent reduced SSBNF grammar G satisfying $|G| \leq q_{\mathrm{size}}(|G_*|)$ and $\tau_G \leq q_{\mathrm{thick}}(|G_*|, \tau_{G_*} + 1)$.*

The proof is in Appendix A.2; terminal isolation and binarization precede non-start λ-elimination so that grammar size and shortest nonempty yields remain polynomially bounded.

Lemma 7.4 (locally trivial–group divisors and window refinements)**.** *Let $h : \Sigma^* \to M$ be a finite-monoid morphism with Σ nonempty. The following are equivalent:*

(i) *$h(\Sigma^+)$ is a homomorphic image of a subsemigroup of $\mathsf{S} \times \mathsf{G}$ for some finite locally trivial semigroup S and finite group G.*

(ii) *There are $k, \ell \geq 0$ and a morphism $g : \Sigma^* \to \mathsf{G}$ into a finite group such that $\ker(h_{k,\ell} \times g)|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$.*

Proof. Assume (i). Choose a subsemigroup $\mathsf{T} \subseteq \mathsf{S} \times \mathsf{G}$ and a surjective semigroup morphism $\pi : \mathsf{T} \twoheadrightarrow h(\Sigma^+)$. For each $a \in \Sigma$, choose $(s_a, q_a) \in \mathsf{T}$ with $\pi(s_a, q_a) = h(a)$. Extend $a \mapsto (s_a, q_a)$ multiplicatively to $\varphi : \Sigma^+ \to \mathsf{T}$. Projecting to the two factors, and assigning the respective identities to λ, gives monoid morphisms $s : \Sigma^* \to \mathsf{S}^1$ and $g : \Sigma^* \to \mathsf{G}$, where S^1 adjoins an identity only if needed. For every $w \in \Sigma^+$, $h(w) = \pi(s(w), g(w))$. As $s(\Sigma^+)$ is a subsemigroup of the locally trivial semigroup S, Proposition 3.5 supplies k, ℓ with $\ker h_{k,\ell}|_{\Sigma^+} \subseteq \ker s|_{\Sigma^+}$, which implies (ii).

Conversely, (ii) induces a well-defined surjective semigroup morphism

$$(h_{k,\ell} \times g)(\Sigma^+) \longrightarrow h(\Sigma^+), \qquad (h_{k,\ell}(w), g(w)) \longmapsto h(w).$$

By Proposition 3.5, $h_{k,\ell}(\Sigma^+)$ is locally trivial, so the domain is a subsemigroup of a product of a locally trivial finite semigroup and a finite group. Thus (i) holds. □

Corollary 7.5 (ordinary-thickness bounds for locally trivial–group typings)**.** *Fix $h : \Sigma^* \to M$. Suppose $h(\Sigma^+)$ is a homomorphic image of a subsemigroup of the direct product of a finite locally trivial semigroup and a finite group. Then every nonempty $L \in \mathcal{C}_h^{\mathrm{cf}}$ represented by a reduced CFG G_* has characteristic data for Rec_h of encoded size polynomial in $|G_*|$ and $\tau_{G_*} + 1$. This includes all locally trivial positive-image typings and all finite-group typings.*

Proof. By Lemma 7.4, Theorem 7.2 applies to the SSBNF grammar given by Proposition 7.3; compose the polynomials. The target $\{\lambda\}$ is handled separately. □

Let **LI** and **Grp** be the pseudovarieties of finite locally trivial semigroups and of finite groups. As both are closed under finite direct products, their join $\mathbf{LI} \vee \mathbf{Grp}$ consists exactly of the divisors (homomorphic images of subsemigroups) of products $\mathsf{S} \times \mathsf{G}$ with $\mathsf{S} \in \mathbf{LI}$ and $\mathsf{G} \in \mathbf{Grp}$; such joins were studied by Costa [22, 23]. The hypothesis of Corollary 7.5 thus reads $h(\Sigma^+) \in \mathbf{LI} \vee \mathbf{Grp}$.

Lemma 7.6 (the two-element semilattice lies outside the window–group join)**.** $U_1 \notin \mathbf{LI} \vee \mathbf{Grp}$.

Proof. Suppose there were a subsemigroup $\mathsf{T} \subseteq \mathsf{S} \times \mathsf{G}$ and a surjective morphism $\pi : \mathsf{T} \to U_1$, with $\mathsf{S} \in \mathbf{LI}$ and G a finite group. Choose idempotents $e \in \pi^{-1}(1)$ and $f \in \pi^{-1}(\zeta)$ (Section 2.2). Every idempotent of $\mathsf{S}\times\mathsf{G}$ has group component 1_{G}, so the S-components of e, f are idempotents. Local triviality of S gives $efe = e$. Applying π would yield $1 \cdot \zeta \cdot 1 = 1$, contradicting $\zeta \neq 1$. □

Remark 7.7. The positive image of h_{U} is U_1, which lies outside $\mathbf{LI} \vee \mathbf{Grp}$ by Lemma 7.6. Proposition 6.3 shows that the ordinary-thickness characteristic-data bound in Corollary 7.5 can fail for Rec_h when its algebraic hypothesis is removed. This does not establish that the hypothesis is necessary for every fixed typing. Proposition 6.3(iv) does give a reconstruction-operator-independent lower bound on characteristic-sample size for the distinguishing target pair, but not a lower bound on the running time or convergence stage of arbitrary Gold learners.

These polynomial bounds may depend on the fixed typing and alphabet; no uniform bound for variable windows or groups is claimed.

8 The Linear Subclass

Linear languages are exactly the languages accepted by one-turn pushdown automata [24, 25]. The normalization below gives the grammatical counterpart: every successful derivation has one continuing nonterminal path, which we call its *spine*.

Proposition 8.1 (linear-spine SSBNF normalization)**.** *There is a fixed polynomial q_{lin} such that every linear CFG G generating a nonempty language can be converted in polynomial time into an equivalent reduced SSBNF grammar $G_0 = (V_0, \Sigma, P_0, S_0)$ with $|G_0| \le q_{\mathrm{lin}}(|G|)$ in which every binary production has exactly one fresh terminal-wrapper child W_a with $W_a \to a$ and exactly one other nonterminal child.*

The construction is given in Appendix B. Let $\widetilde{G}_0$ be its typed refinement and $N_t := |\widetilde{V}_0 \setminus \{\widetilde{S}\}| \le |V_0 \setminus \{S_0\}|\,|M|$.

Lemma 8.2 (short canonical yields)**.** *Every typed non-start symbol X of $\widetilde{G}_0$ satisfies $|\omega(X)| \le N_t$. Hence $\tau_h^{\mathrm{typ}}(G_0) \le N_t$.*

Proof. By Proposition 8.1, each binary spine step $A \to W_aB$ or $A \to BW_a$ emits exactly one terminal outside the continuing nonterminal. If X is a terminal-wrapper symbol then $|\omega(X)| = 1$. Otherwise choose a shortest terminal derivation from the typed spine symbol X and write its spine as $X = X_0, X_1, \ldots, X_t$, where X_t uses a terminal rule. Its yield has length $t + 1$. No typed spine symbol can repeat: if $X_i = X_j$ with $i < j$, deleting the segment between the two occurrences gives a valid derivation with fewer spine steps and hence a shorter yield. Thus the $t + 1$ typed spine symbols are distinct non-start symbols, so $t + 1 \le N_t$. Hence $|\omega(X)| \le N_t$ in all cases. □

Theorem 8.3 (polynomial characteristic data for linear targets)**.** *For fixed h, every $L \in \mathcal{C}_h^{\mathrm{lin}}$ represented by a linear CFG G has a characteristic sample for Rec_h whose $\|\cdot\|$-size is polynomial in $|G|$.*

Proof. For $L = \emptyset$ and $L = \{\lambda\}$ use the samples from Section 5. Otherwise take G_0 from Proposition 8.1. Lemma 8.2 gives $\tau_h^{\mathrm{typ}}(G_0) \le N_t \le |V_0 \setminus \{S_0\}|\,|M|$, and Corollary 6.2 yields a characteristic sample polynomial in $|G_0| \le q_{\mathrm{lin}}(|G|)$. More explicitly, the bound in Lemma 6.1 gives $\|\mathsf{Wit}(\widetilde{G}_0)\| = O(|G_0|^3|M|^4)$; the polynomial dependence on the original $|G|$ also uses the normalization bound q_{lin}. □

Together with Theorem 5.6, this gives polynomial-time batch reconstruction and polynomial characteristic data relative to a chosen linear CFG for targets in $\mathcal{C}_h^{\mathrm{lin}}$. This does not contradict de la Higuera's negative result for the unrestricted class of linear languages [18, Theorem 2]: our theorem concerns only the fixed-h substitutable subclass. Moreover, no incremental polynomial-data guarantee for Learn_h is asserted.

A nonregular linear fixed-h example. The preceding theorem is not confined to regular targets, nor to the Clark–Eyraud or fixed-window classes. Define

$$L_{\pm,e} := \{a^{2m}cb^{2m} : m \ge 0\} \cup \{a^{2m+1}db^{2m+1} : m \ge 0\} \cup \{a^n eb^n : n \ge 0\}.$$

It is generated by the linear grammar

$$S \to U_{\mathrm{ev}} \mid U_e, \quad U_{\mathrm{ev}} \to c \mid aU_{\mathrm{odd}}b, \quad U_{\mathrm{odd}} \to d \mid aU_{\mathrm{ev}}b, \quad U_e \to e \mid aU_eb.$$

The subscripts ev and odd track the parity of enclosing a, b-pairs from the start, not the length parity of the corresponding nonterminal's own yield.

Call c, d, e the *center symbols*.

Proposition 8.4. *The language $L_{\pm,e}$ is linear, nonregular, and belongs to $\mathcal{C}_h^{\mathrm{lin}}$ for some homomorphism h into a finite monoid. It is neither Clark–Eyraud substitutable nor (k,ℓ)-substitutable for any fixed $k, \ell \ge 0$.*

Proof. Linearity follows from the displayed grammar. To establish finite-typed substitutability, put

$$L_{\mathrm{all}} := \{a^n \xi b^n : n \ge 0,\ \xi \in \{c, d, e\}\}, \qquad R_{\mathrm{reg}} := (aa)^*c(bb)^* \cup a(aa)^*d\,b(bb)^* \cup a^*eb^*.$$

The linear language L_{all} is Clark–Eyraud substitutable under the nonempty-factor convention: a shared context uniquely determines a center-free factor, while the distribution of a factor $a^i\xi b^j$ containing one center depends only on $i-j$. Thus $L_{\mathrm{all}} \in \mathcal{C}_{h_0}^{\mathrm{lin}}$ for the trivial typing h_0. Since R_{reg} is regular and $L_{\pm,e} = L_{\mathrm{all}} \cap R_{\mathrm{reg}}$, Proposition 3.2(ii), together with closure of linear languages under regular intersection, gives $L_{\pm,e} \in \mathcal{C}_{h_0\times g}^{\mathrm{lin}}$ for a finite-monoid homomorphism g recognizing R_{reg}.

Moreover, $L_{\pm,e} \cap a^*eb^* = \{a^n eb^n : n \ge 0\}$ proves nonregularity. The factors c, e share the empty context, but $(a, b) \in \mathcal{D}_{L_{\pm,e}}(e) \setminus \mathcal{D}_{L_{\pm,e}}(c)$; hence $L_{\pm,e}$ is not Clark–Eyraud substitutable. Finally, fix $k, \ell \ge 0$ and put $m_0 := \max\{1, k+\ell\}$, $w_e := a^k eb^\ell$, and $w_c := a^k cb^\ell$. The factors w_e, w_c have the same length-k prefix and length-ℓ suffix and share the context $(a^{2m_0-k}, b^{2m_0-\ell})$, whereas $(a^{2m_0+1-k}, b^{2m_0+1-\ell}) \in \mathcal{D}_{L_{\pm,e}}(w_e) \setminus \mathcal{D}_{L_{\pm,e}}(w_c)$. The fixed-window counterexample criterion gives $L_{\pm,e} \notin \mathsf{KL}$. □

A concrete four-element typing. The regular-filter argument establishes the required existence of a finite typing. In fact, a four-element typing already suffices. Let $M_{\pm,e} := \{1, \kappa_{cd}, \kappa_e, 0\}$ be the monoid with identity 1 and $\mu\nu = 0$ for all $\mu, \nu \in \{\kappa_{cd}, \kappa_e, 0\}$, and define $h_{\pm,e} : \{a, b, c, d, e\}^* \to M_{\pm,e}$ by

$$h_{\pm,e}(a) = h_{\pm,e}(b) = 1, \qquad h_{\pm,e}(c) = h_{\pm,e}(d) = \kappa_{cd}, \qquad h_{\pm,e}(e) = \kappa_e. \tag{3}$$

Then $L_{\pm,e}$ is $\sim_{h_{\pm,e}}$-substitutable. Indeed, two center-free nonempty factors sharing a context must coincide. The distribution of $a^i e b^j$ depends only on $i-j$, while that of $a^i \xi b^j$ for $\xi \in \{c,d\}$ depends only on $(i-j, \mathrm{par}(\xi) - i \bmod 2)$, where $\mathrm{par}(c)=0$ and $\mathrm{par}(d)=1$. A shared context forces these invariants to agree, and equal $h_{\pm,e}$-types distinguish e-factors from c/d-factors; thus equal-type factors with a common context have the same distribution.

Thus Theorem 8.3 applies to a nonregular linear language outside KL. The displayed grammar is even-linear, so no separation from Takada's control-set framework is claimed.

Finite-state control induced by yield typing. The following straightforward product construction gives a limited connection with Takada's control-set viewpoint. Give each production of a linear grammar $G = (V, \Sigma, P, S)$ a distinct label from $\mathsf{ProdLab}$, and write $A \Rightarrow_G^{\alpha} w$ when $\alpha \in \mathsf{ProdLab}^*$ is the production-label word of a derivation. For $\mathsf{Acc}_{\mathrm{ctrl}} \subseteq M$, let

$$\mathsf{Ctrl}_{h,\mathsf{Acc}_{\mathrm{ctrl}}}(G) := \{\alpha \in \mathsf{ProdLab}^* : S \Rightarrow_G^{\alpha} w \text{ for some } w \in \Sigma^*,\ h(w) \in \mathsf{Acc}_{\mathrm{ctrl}}\}.$$

This is a regular control set. A finite automaton uses states $V \times M \times M$ (and one accepting state), starts in $(S, 1, 1)$, and for a labeled rule $A \to uBv$ makes the transition $(A, \mu_L, \mu_R) \mapsto (B, \mu_L h(u), h(v)\mu_R)$. A terminal rule $A \to w$ is accepted exactly when $\mu_L h(w) \mu_R \in \mathsf{Acc}_{\mathrm{ctrl}}$. The invariant is that a sentential form uAv is represented by $(A, h(u), h(v))$, so the accepted label words are exactly those of successful derivations whose yields have h-type in $\mathsf{Acc}_{\mathrm{ctrl}}$. Thus regular control by $\mathsf{Ctrl}_{h,\mathsf{Acc}_{\mathrm{ctrl}}}(G)$ generates precisely $L(G) \cap h^{-1}(\mathsf{Acc}_{\mathrm{ctrl}})$. Starting from any nonterminal A with $\mathsf{Acc}_{\mathrm{ctrl}} = \{\mu\}$ gives the same interpretation for a typed copy A_μ. This is a finite-state bridge to Takada's regular control languages [14]; it does not claim that arbitrary Takada control sets arise from terminal yield typings.

9 Expressiveness and Boundaries

We now locate the framework between the fixed-window hierarchy and broader context-free families.

9.1 A nonlinear target inside RS

Let $\Delta := \{a^n b^n : n \geq 0\}$ and consider Δ^*. For $z \in \{a,b\}^*$ put $\beta(z) := |z|_a - |z|_b$; the *height* after reading a prefix p is $\beta(p)$. For any word x, define its *admissible entry-height set* by

$$\mathcal{Q}(x) := \{q \in \mathbb{Z}_{\geq 0} : q + \beta(x) \geq 0, \quad a^q x b^{q+\beta(x)} \in \Delta^*\}.$$

A useful local characterization is that $w \in \Delta^*$ if and only if all prefix heights of w are nonnegative, $\beta(w) = 0$, and immediately after the b in every occurrence of ba the height is zero. Indeed, the ba boundaries split w into blocks of the form $a^i b^j$; the stated conditions force each block to start and end at height zero and hence to have $i = j$. Conversely, every concatenation of such blocks has the stated properties. Applying this characterization to the canonical completion $a^q x b^{q+\beta(x)}$ (whose boundaries with x introduce no new ba factor) shows that $\mathcal{Q}(x)$ is exactly the set of heights $\beta(u)$ at which x occurs in a word $uxv \in \Delta^*$. For the converse direction, reading x from height $\beta(u)$ in such a word preserves all internal nonnegativity and ba-boundary conditions; its canonical completion therefore belongs to Δ^*.

For example, $\mathcal{Q}(ab) = \mathbb{Z}_{\geq 0}$, $\mathcal{Q}(aa) = \mathbb{Z}_{\geq 0}$, $\mathcal{Q}(\lambda) = \mathbb{Z}_{\geq 0}$, $\mathcal{Q}(bb) = \mathbb{Z}_{\geq 2}$, $\mathcal{Q}(ba) = \{1\}$, $\mathcal{Q}(bbaa) = \{2\}$, and $\mathcal{Q}(baaba) = \varnothing$.

The canonical completion gives the membership criterion

$$w \in \Delta^* \iff \beta(w) = 0 \text{ and } 0 \in \mathcal{Q}(w). \tag{$\star$}$$

Let $\mathrm{fst}(w), \mathrm{lst}(w) \in \{\bot, a, b\}$ be the first and last symbols, with $\bot$ on λ, and let $\mathrm{ba}(w)$ record whether w contains ba. Put $h_\star(w) := (\mathrm{fst}(w), \mathrm{lst}(w), \mathrm{ba}(w))$. The summary of a concatenation

is determined by the summaries of its factors: the endpoints concatenate in the evident way and a new ba can arise only across a boundary $b|a$. Hence the finite image $M_\star := h_\star(\{a,b\}^*)$ carries the product $h_\star(x)h_\star(y) := h_\star(xy)$, making $h_\star$ a finite-monoid homomorphism.

Appending a word on the right cannot enlarge the admissible entry-height set; more precisely, the following concatenation identity holds:[2]

$$\mathcal{Q}(st) = \mathcal{Q}(s) \cap \{q \in \mathbb{Z} : q + \beta(s) \in \mathcal{Q}(t)\} \cap \begin{cases} \{-\beta(s)\}, & (\mathrm{lst}(s), \mathrm{fst}(t)) = (b,a), \\ \mathbb{Z}, & \text{otherwise.} \end{cases} \tag{$\star\star$}$$

Proposition 9.1 (a nonlinear fixed-$h_\star$ target)**.** *The language Δ^* is deterministic context-free, nonregular, and nonlinear, and it is $\sim_{h_\star}$-substitutable. Moreover it is not (k,ℓ)-substitutable for any fixed $k,\ell \geq 0$. Hence $\Delta^* \in \mathsf{RS}_{h_\star}(\{a,b\}) \cap \mathrm{CFL}$ and $\Delta^* \in \mathsf{RS}(\{a,b\}) \setminus \mathsf{KL}(\{a,b\})$.*

Proof. The grammar $S \to TS \mid \lambda$, $T \to aTb \mid \lambda$ generates Δ^*. It is deterministic context-free: a DPDA pushes while reading the a-part of a factor $a^n b^n$, pops during its b-part, and permits a new such factor only after the stack returns to its bottom marker. Nonregularity follows from $\Delta^* \cap a^*b^* = \Delta$, and nonlinearity from closure of linear languages under regular intersection together with $\Delta^* \cap a^*b^*a^*b^* = \Delta\Delta$. The nonlinearity of $\Delta\Delta$ is explicitly recorded by Autebert–Berstel–Boasson [25, Section 6.1, before Theorem 6.1].

For substitutability, let nonempty x, y have equal $h_\star$-type and a common context (u,v). By the preceding characterization of $\mathcal{Q}$ and additivity of β, the memberships $uxv, uyv \in \Delta^*$ imply $\beta(x) = \beta(y)$ and $\beta(u) \in \mathcal{Q}(x) \cap \mathcal{Q}(y)$. Since $h_\star(x) = h_\star(y)$, either both contain ba or neither does.

In the former case, write $x = rbat$ with $r, t \in \{a,b\}^*$. In any word of Δ^*, the height immediately after the b in each occurrence of ba is 0. Hence every $q \in \mathcal{Q}(x)$ satisfies $q + \beta(rb) = 0$, so $\mathcal{Q}(x)$ has at most one element; the same holds for $\mathcal{Q}(y)$. Since $\beta(u) \in \mathcal{Q}(x) \cap \mathcal{Q}(y)$, we obtain $\mathcal{Q}(x) = \{\beta(u)\} = \mathcal{Q}(y)$.

In the latter case, $x, y \in a^*b^*$, and for each $z \in \{x, y\}$ we have $\mathcal{Q}(z) = \{q \in \mathbb{Z} : q \geq -\min\{0, \beta(z)\}\}$. Since we have already shown $\beta(x) = \beta(y)$, it follows immediately that $\mathcal{Q}(x) = \mathcal{Q}(y)$.

Thus $\beta(x) = \beta(y)$ and $\mathcal{Q}(x) = \mathcal{Q}(y)$. Also, $\mathrm{fst}(x) = \mathrm{fst}(y)$ and $\mathrm{lst}(x) = \mathrm{lst}(y)$ by equality of $h_\star$-types. Applying ($\star\star$) twice gives, for all u', v', $\beta(u'xv') = \beta(u'yv')$ and $\mathcal{Q}(u'xv') = \mathcal{Q}(u'yv')$. By ($\star$), $u'xv' \in \Delta^*$ iff $u'yv' \in \Delta^*$. Thus $\mathcal{D}_{\Delta^*}(x) = \mathcal{D}_{\Delta^*}(y)$.

For the fixed-window separation, fix k, ℓ and let $m := \max\{1, k+\ell\}$, $x := a^m b^m$, and $y := a^m b^m a^m b^m$. They have the same length-k prefix and length-ℓ suffix and share the empty context. Moreover $axb \in \Delta^*$, whereas the central ba in ayb has height 1, so $ayb \notin \Delta^*$. The fixed-window counterexample criterion therefore excludes every (k,ℓ). □

9.2 A capped regular counter family

Yoshinaka's regular language $L_{\mathrm{Yreg}} = ae^*ce^*a \cup ae^*de^*a \cup be^*ce^*b$ already gives a fixed-window separation [3, Proposition 1]; capped counters contrast with the uncapped obstruction below.

Let $\Sigma_c := \{\uparrow, \downarrow, \#\}$. For $w = \delta_1 \cdots \delta_n \in \Sigma_c^*$, let $\mathrm{ht}_i(w)$ be the height after i letters, with $\mathrm{ht}_0(w) = 0$, changing by $+1$ on $\uparrow$, -1 on $\downarrow$, and resetting to 0 on $\#$. For $\rho \geq 1$, put

$$\mathrm{CTR}_\rho := \{w \in \Sigma_c^* : 0 \leq \mathrm{ht}_i(w) \leq \rho \text{ for all } 0 \leq i \leq |w|\}.$$

For every $\rho \geq 1$, CTR_ρ is regular: a deterministic finite automaton records heights $0, \ldots, \rho$, uses one sink outside the range, and resets to 0 on $\#$. Hence $\mathrm{CTR}_\rho \in \mathsf{RS}(\Sigma_c)$ by Proposition 3.1.

[2] Indeed, the conditions for reading st from height q split into those within s, namely $q \in \mathcal{Q}(s)$; those within t, namely $q + \beta(s) \in \mathcal{Q}(t)$; and the condition at the boundary $s|t$. Only when this boundary is $b|a$ must the height immediately after that b be 0, requiring $q + \beta(s) = 0$. These conditions are also sufficient, and the identity follows.

Theorem 9.2. *For every $\rho \geq 2$, CTR_ρ is not (k,ℓ)-substitutable for any $k, \ell \geq 0$.*

Proof. Fix k, ℓ and put $x := (\uparrow\downarrow)^k \uparrow^{\rho-1} \#^\ell$ and $y := (\uparrow\downarrow)^k \uparrow^\rho \#^\ell$. They are long enough for the fixed-window criterion, have the same length-k prefix and length-ℓ suffix, and both lie in CTR_ρ. Yet $\uparrow x \in \mathrm{CTR}_\rho$ while $\uparrow y \notin \mathrm{CTR}_\rho$, the latter reaching height $\rho + 1$. Hence their distributions differ, and the criterion applies. □

The bound $\rho \geq 2$ is sharp[3]. The DFA transition monoid gives a typing h_ρ recognizing each regular CTR_ρ; only $\rho \geq 2$ lies outside the fixed-window hierarchy.

9.3 Natural deterministic context-free languages outside RS

Lemma 9.3 (finite-monoid obstruction)**.** *Let $L \subseteq \Sigma^*$. Suppose an infinite $\Xi \subseteq \Sigma^+$ satisfies, for all distinct $x, y \in \Xi$, $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ and $\mathcal{D}_L(x) \neq \mathcal{D}_L(y)$. Then $L \notin \mathsf{RS}(\Sigma)$.*

Proof. Any finite h identifies two elements of Ξ; their common context and unequal distributions contradict $\sim_h$-substitutability. □

Removing the upper cap gives the deterministic context-free language

$$\mathrm{CTR} := \{w \in \Sigma_c^* : \mathrm{ht}_i(w) \geq 0 \text{ for all } 0 \leq i \leq |w|\},$$

accepted by the evident deterministic pushdown counter with reset.

Corollary 9.4 (uncapped counter obstruction)**.** $\mathrm{CTR} \notin \mathsf{RS}(\Sigma_c)$.

Proof. Use $\Xi = \{\uparrow^i : i \geq 1\}$. For $i < j$, $(\lambda, \downarrow^i)$ is common, while $(\lambda, \downarrow^j)$ is a context only of $\uparrow^j$. □

For the one-bracket Dyck language

$$D_1 := \{w \in \{a,b\}^* : |w'|_a \geq |w'|_b \text{ for every prefix } w',\ |w|_a = |w|_b\},$$

This is a standard deterministic context-free language [27]. Here D_1 denotes the whole Dyck language, written D_1^* in the convention of Autebert–Berstel–Boasson [25].

Corollary 9.5. $D_1 \notin \mathsf{RS}(\{a,b\})$.

Proof. Use $\Xi = \{b^i a^i : i \geq 1\}$. For $i < j$, (a^j, b^j) is common, whereas (a^i, b^i) is a context only of $b^i a^i$; for $b^j a^j$ the prefix $a^i b^j$ underflows. □

Thus unbounded stack behavior alone is not the obstruction: the nonregular one-turn language $L_{\pm,e}$ lies in RS, but an infinite family of mutually comparable factors with distinct contextual behavior cannot be separated by any finite h.

Lemma 9.6 (fixed-word quotient)**.** *If L is $\sim_h$-substitutable and $z \in \Sigma^*$, then $L/z := \{w : wz \in L\}$ is $\sim_h$-substitutable.*

Proof. A common context (u, v) in L/z becomes (u, vz) in L; equality of distributions in L then transfers every context (s, t) by appending z. □

The language generated by $S \to aSS \mid b$ is the *Łukasiewicz language*. Yoshinaka [3, Section 3] already observed that this language is not (k,ℓ)-substitutable for any fixed k, ℓ. In the notation used here, $L_{\mathrm{Luk}} = D_1 b$ [25], and it is deterministic context-free [27]. The quotient argument gives the stronger conclusion that it lies outside the present finite-typing union: if $L_{\mathrm{Luk}} \in \mathsf{RS}(\{a,b\})$, then Lemma 9.6 would put $D_1 = L_{\mathrm{Luk}}/b$ in RS, contradicting Corollary 9.5.

[3]Indeed, CTR_1 is $(1,1)$-substitutable. Membership is characterized by the initial symbol not being $\downarrow$ and the absence of the length-2 factors $\uparrow\uparrow$, $\downarrow\downarrow$, and $\# \downarrow$. Thus, for nonempty factors occurring in words of CTR_1 with a common context, equality of their first and last symbols ensures equal membership under every context.

9.4 Comparison with Clark's congruential family

Clark [28] allows a finite initial set $I \subseteq V$; write $L_G[I] := \bigcup_{A \in I} L_G(A)$. Such a grammar is *congruential* if each $L_G(A)$ lies in one syntactic-congruence class of $L_G[I]$. Let $\mathsf{CONG}(\Gamma)$ be the languages over Γ generated by such grammars. Clark already places substitutable and (k,ℓ)-substitutable languages in this family; the following extends the containment to every fixed-h slice.

Proposition 9.7 (comparison with Clark's congruential family)**.** *For every finite alphabet* Γ *and finite-monoid homomorphism* $h : \Gamma^* \to M$, $\mathcal{C}_h^{\mathrm{cf}} \subseteq \mathsf{CONG}(\Gamma)$. *Consequently* $\mathsf{RS}(\Gamma) \cap \mathrm{CFL} \subseteq \mathsf{CONG}(\Gamma)$, *and for* $\Gamma = \{a, b\}$ *the inclusion is proper.*

Proof. Let nonempty $L \in \mathcal{C}_h^{\mathrm{cf}}$ and take its reduced SSBNF grammar and typed refinement $\widetilde{G}$. Every non-start typed symbol $X = A_\mu$ has a terminal reaching context (u, v), and every $w \in L_{\widetilde{G}}(X)$ is nonempty with $h(w) = \mu$ by Proposition 5.2. Hence any $x, y \in L_{\widetilde{G}}(X)$ have the same type and the common context (u, v); fixed-h substitutability gives $x \equiv_L y$. Thus each typed nonterminal generates strings from one syntactic-congruence class.

Delete the separated start symbol and use its typed successors as the finite initial set; if $\lambda \in L$, add one fresh initial symbol generating only λ. Proposition 5.2 then gives a congruential grammar for L. The empty language is handled by an initial symbol with no productions. This proves the inclusion after taking the union over h.

For properness over $\{a, b\}$, use the Dyck example also discussed by Clark [28]. Every $x \in D_1$ satisfies $uxv \in D_1$ iff $uv \in D_1$, so $x \equiv_{D_1} \lambda$. Hence $S \to aSbS \mid \lambda$ is congruential, while Corollary 9.5 gives $D_1 \notin \mathsf{RS}(\{a, b\})$. □

10 Conclusion

Finite-monoid typing brings context-free targets beyond fixed-window substitutability into positive-data learning, but refinement can make characteristic data exponentially larger. Our results separate polynomial-time reconstruction and conservative Gold identification from quantitative data guarantees. The exponential lower bounds persist across all reduced SSBNF presentations in typed thickness and extend to every set-driven operator admitting characteristic samples for both targets of a distinguishing pair.

The polynomial characteristic-data bounds established here hold under stated sufficient algebraic conditions or for linear targets. We claim neither necessity of those conditions nor polynomial bounds in grammar size and ordinary thickness for arbitrary fixed typings. The lower bounds do not concern the running time or convergence stage of arbitrary Gold learners. Characterizing $\mathsf{RS} \cap \mathrm{CFL}$ remains open.

Data availability

No empirical data were used for the research described in this article.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Acknowledgments

The author thanks Makoto Kanazawa and Ryo Yoshinaka for guidance, encouragement, and help with relevant literature, and the two anonymous reviewers for careful and constructive reports that substantially improved the paper, in particular by identifying problems in the submitted learning algorithm and by prompting the direct two-word witness in Theorem 9.2. Any remaining errors are the author's responsibility.

A Fixed-window witness bounds and thickness-preserving normalization

A.1 Proof of Lemma 7.1

Proof of Lemma 7.1. Put $m := k + \ell$, $q_{\mathrm{grp}} := |\mathsf{G}|$, $N := |V \setminus \{S_0\}|$, $D_{\mathrm{bd}} := (m+1)Nq_{\mathrm{grp}}$, and $W_{\mathrm{bd}} := m + q_{\mathrm{grp}} - 1$. Every non-start symbol of the reduced SSBNF grammar G derives a nonempty word of length at most τ_G, so $\tau_G \geq 1$.

Fiber and shortening. Fix a retained non-start A_μ of $\widetilde{G}$. By Proposition 5.2, choose $w_0 \in L_{\widetilde{G}}(A_\mu) = L_G(A) \cap h^{-1}(\{\mu\})$ and put $\eta := (h_{k,\ell}(w_0), g(w_0))$. The kernel inclusion gives

$$L_G(A) \cap (h_{k,\ell} \times g)^{-1}(\{\eta\}) \subseteq L_{\widetilde{G}}(A_\mu).$$

The left side contains w_0. Let w be its shortest word; then $|\omega(A_\mu)| \leq |w|$. If $|w| < m$, the bound follows. Otherwise write $w = \alpha\beta\gamma$, with $|\alpha| = k$ and $|\gamma| = \ell$, and protect the m positions of α, γ, denoting their set by $\mathsf{GuardPos}$ (possibly empty). Any $w' = \alpha\beta'\gamma \in L_G(A)$ with $g(w') = g(w)$ has $|w'| \geq |w|$: all such yields are nonempty and have the same $(h_{k,\ell} \times g)$-type as w. We use this shortest-fiber property throughout.

Fix a derivation tree of $A \Rightarrow_G^* w$, taking nonterminal occurrences as nodes and lexical rules $B \to a$ as leaves. For a node z, let $\mathrm{nt}(z)$ be its non-start label, $y(z)$ its nonempty yield, $\mathrm{span}(z)$ its yield interval, and $\mathrm{Mark}(z) := \mathrm{span}(z) \cap \mathsf{GuardPos}$. Subtrees at incomparable nodes may be replaced independently by derivations from their respective labels.

Depth. The triple $(\mathrm{Mark}(z), \mathrm{nt}(z), g(y(z)))$ cannot repeat along an ancestor chain: replacing the higher subtree by the lower one preserves the protected letters and group value but strictly shortens the yield (each proper descendant lies below a binary branching with a nonempty sibling yield). Since the two marks coincide, all deleted positions avoid $\mathsf{GuardPos}$, so the new yield retains prefix α and suffix γ. This contradicts shortestness. Along any root-to-leaf path, the nested sets $\mathrm{Mark}(z)$ take at most $m+1$ values; labels and group values have at most N and q_{grp} choices. Hence every node has depth less than D_{bd}.

Width. Call z *essential* if $\mathrm{Mark}(z) \neq \varnothing$ or $|y(z)| > \tau_G$, and let Ess be the set of essential nodes. At any fixed depth, at most m nodes of Ess meet $\mathsf{GuardPos}$. If a depth contains at least $m + q_{\mathrm{grp}}$ essential nodes, at least q_{grp} of them do not meet $\mathsf{GuardPos}$; choose $z_1, \ldots, z_{q_{\mathrm{grp}}}$ among these, ordered from left to right. Their disjoint yields $a_i := y(z_i)$ lie inside β and satisfy $|a_i| > \tau_G$. Choose $b_i \in L_G(\mathrm{nt}(z_i))$ with $|b_i| \leq \tau_G$, and write $w = c_0 a_1 c_1 \cdots a_{q_{\mathrm{grp}}} c_{q_{\mathrm{grp}}}$. For $1 \leq j \leq q_{\mathrm{grp}}$, set

$$r_j := g(a_1 c_1 \cdots a_j c_j), \quad s_j := g(b_1 c_1 \cdots b_j c_j), \quad x_j := s_j r_j^{-1},$$

with $r_0 = s_0 = x_0 = 1_{\mathsf{G}}$. Among $x_0, \ldots, x_{q_{\mathrm{grp}}}$ two coincide, say $x_i = x_j$ for $i < j$. Put $r_{i:j} := g(a_{i+1} c_{i+1} \cdots a_j c_j)$ and $s_{i:j} := g(b_{i+1} c_{i+1} \cdots b_j c_j)$. Since $r_j = r_i r_{i:j}$ and $s_j = s_i s_{i:j}$, $s_i r_i^{-1} = s_i s_{i:j} r_{i:j}^{-1} r_i^{-1} \Longrightarrow s_{i:j} = r_{i:j}$. Replacing $z_{i+1}, \ldots, z_j$ by derivations of the corresponding b-words therefore preserves $g(w)$ and the protected positions, but strictly shortens w, a contradiction. This uses no commutativity of G. Thus each depth contains at most $W_{\mathrm{bd}} = m + q_{\mathrm{grp}} - 1$ essential nodes.

Count. As Ess is ancestor-closed, $|\mathsf{Ess}| \leq W_{\mathrm{bd}} D_{\mathrm{bd}}$. Every unprotected position lies below a nonessential lexical leaf (of yield length $1 \leq \tau_G$). The maximal nonessential subtrees are pairwise

disjoint, each has yield length at most τ_G, and their roots number at most $2|\mathsf{Ess}|$ if $\mathsf{Ess} \neq \varnothing$, or one if $\mathsf{Ess} = \varnothing$. Consequently,

$$|w| \le m + (1 + 2|\mathsf{Ess}|)\tau_G \le m + \big[1 + 2(m + q_{\mathrm{grp}} - 1)(m+1)Nq_{\mathrm{grp}}\big]\tau_G.$$

Taking the maximum over retained A_μ proves the lemma. □

A.2 Thickness-preserving SSBNF normalization

Proof of Proposition 7.3. Let $G_* = (V, \Sigma, P, S)$ be reduced with $L(G_*) \neq \emptyset$, and put $s_* := |G_*|$ and $\bar{\tau} := \tau_{G_*} + 1$. The standard CFG normalization operations are classical [25, Section 3.1] [26, Sections 7.1 and 7.4.2]. We specify their order and the shortest-yield estimates required here; the latter do not follow merely from existence of a normal form.

Introduce a fresh start symbol $S_0 \to S$, retaining $S_0 \to \lambda$ precisely when $\lambda \in L(G_*)$. First isolate terminals in longer right-hand sides using $W_a \to a$, and then binarize. The resulting G_{bin} has $O(s_*)$ nonterminals and productions in symbol-count size (and polynomial encoded size); original nonterminals retain their terminal languages. Each fresh binarization symbol represents a contiguous block of an original right-hand side with $O(s_*)$ symbols. Replacing these symbols by shortest terminal yields, each of length at most $\bar{\tau}$, gives $\tau_{G_{\mathrm{bin}}} = O(s_*\bar{\tau})$.

For any nonterminal A deriving a nonempty word, mark a terminal leaf in a derivation tree and follow its root-to-leaf path. Whenever a nonterminal repeats on that path, replace the subtree at its higher occurrence by that at its lower occurrence. The marked leaf survives, and the resulting path contains at most $|V_{G_{\mathrm{bin}}}|$ nonterminal occurrences. Replace each off-path sibling by a shortest terminal yield, possibly λ. This still gives a nonempty word and shows

$$\min\{\, |w| > 0 : A \Rightarrow^*_{G_{\mathrm{bin}}} w \,\} \le 1 + |V_{G_{\mathrm{bin}}}|\tau_{G_{\mathrm{bin}}} = O(s_*^2\bar{\tau}).$$

Now eliminate non-start λ-productions. Because binarization has already been performed, each $A \to BC$ yields at most the nonempty-right-hand-side variants $A \to BC$, $A \to B$, and $A \to C$; there is no exponential nullable-subset expansion. Unit-closure elimination then copies at most $O(s_*^2)$ non-unit productions, since there are $O(s_*)$ nonterminals and $O(s_*)$ such productions beforehand. Both eliminations preserve the nonempty terminal language of each non-start symbol, so the preceding bound survives. Finally trim useless symbols, retaining λ only at S_0. The result is an equivalent reduced SSBNF grammar of polynomial encoded size and thickness $O(s_*^2(\tau_{G_*} + 1))$. Every step is computable in polynomial time. □

B Proof of the linear-spine SSBNF normalization

Proof of Proposition 8.1. The standard start-symbol, λ-rule, and unit-rule normalizations are classical [25, Section 3.1] [26, Sections 7.1 and 7.4.2]. The point here is to preserve the one-wrapper-child spine structure. If $L(G) = \{\lambda\}$, the single rule $S_0 \to \lambda$ suffices. Otherwise introduce a fresh start symbol $S_0 \to S$, and add $S_0 \to \lambda$ exactly when $\lambda \in L(G)$. Trim useless symbols, eliminate non-start λ-rules, and then eliminate non-start unit rules, without changing the start rules. Linearity ensures at most one additional nonempty right-hand side per rule during λ-elimination; unit-closure elimination also preserves polynomial size and linearity.

For each $a \in \Sigma$, introduce a fresh *wrapper* $W_a \to a$, disjoint from all existing nonterminals. All other new nonterminals below are *spine symbols*, chosen afresh for each production except at the indicated endpoints. For a remaining rule $A \to a_1 \cdots a_m B b_1 \cdots b_n$, where $m + n > 0$, form the right and left spine chains

$$\begin{aligned} C_0^{\mathrm{R}} &:= B, \quad & C_j^{\mathrm{R}} &\to C_{j-1}^{\mathrm{R}} W_{b_j} && (1 \le j \le n),\\ C_0^{\mathrm{L}} &:= C_n^{\mathrm{R}}, \quad & C_i^{\mathrm{L}} &\to W_{a_{m-i+1}} C_{i-1}^{\mathrm{L}} && (1 \le i \le m). \end{aligned}$$

Identify $C_n^{\mathrm{R}} = A$ if $m = 0$, and $C_m^{\mathrm{L}} = A$ otherwise; a zero-length chain contributes no rules. These chains derive exactly the original right-hand side.

For a terminal-only rule $A \to a_1 \cdots a_k$, retain it if $k = 1$; otherwise set $C_1^{\mathrm{T}} := A$, take fresh $C_2^{\mathrm{T}}, \ldots, C_k^{\mathrm{T}}$, and use

$$C_i^{\mathrm{T}} \to W_{a_i} C_{i+1}^{\mathrm{T}} \quad (1 \le i < k), \qquad C_k^{\mathrm{T}} \to a_k.$$

Consequently, each binary rule has exactly one wrapper child, while its other child continues the spine. Final trimming preserves this property and gives an equivalent reduced SSBNF grammar. Since each chain has length linear in the right-hand side it replaces, the construction has polynomial encoded size and running time. □

C Substitutability of the bracketed doubling family

Set $L_0 = \{\mathtt{a}\}$. The languages $L_n = L(G_n)$ from Proposition 6.3 satisfy $L_n = \lceil L_{n-1} \rceil \cup \lfloor L_{n-1} L_{n-1} \rfloor$ for $n \ge 1$, and $L_{G_n}(E_i) = L_i$ for $0 \le i \le n$. Induction on i shows that every word of L_i starts with exactly i opening brackets followed by $\mathtt{a}$. We prove substitutability directly by a deterministic, reversible top-down parser.

Parser and stack properties. Fix $n \ge 1$. A stack is a word over $E_0, \ldots, E_n$ and the closing terminals $\rceil$ and $\rfloor$, with its top on the left. The empty stack is λ. For an arbitrary stack ψ and $1 \le i \le n$, all transitions are

$$E_0\psi \xrightarrow{\mathtt{a}} \psi, \quad E_i\psi \xrightarrow{\lceil} E_{i-1}\rceil\psi, \quad E_i\psi \xrightarrow{\lfloor} E_{i-1}E_{i-1}\rfloor\psi,$$
$$\rceil\psi \xrightarrow{\rceil} \psi, \qquad \rfloor\psi \xrightarrow{\rfloor} \psi.$$

Write $\sigma \xrightarrow{w} \theta$ for the run reading w from σ to θ. These transitions are deterministic. For each fixed input letter they are also injective: its output either reveals the unique expanded E_i and the untouched suffix, or results from deleting a top symbol determined by that input letter. Hence every word acts as a partial injection: if $\sigma \xrightarrow{v} \theta_{\mathrm{out}}$ and $\theta \xrightarrow{v} \theta_{\mathrm{out}}$, then $\sigma = \theta$.

Assign weight 4^i to E_i and weight 1 to either closing marker, and sum over stacks. Every transition strictly decreases weight: the $\mathtt{a}$-transition and both closing-marker pops remove weight 1, while $4^i > 2 \cdot 4^{i-1} + 1 > 4^{i-1} + 1$ for $i \ge 1$. In particular $\sigma \xrightarrow{w} \sigma$ implies $w = \lambda$. Two elementary stack facts will be used. First, in a run from $v\vartheta$ with $v \ne \lambda$, until its first visit to ϑ the stack has the form $v'\vartheta$ with $v' \ne \lambda$ and its moves affect only v'. If it visits ϑ, the run splits as $v \xrightarrow{w_1} \lambda$ followed by $\vartheta \xrightarrow{w_2} \theta$, where $w = w_1 w_2$; otherwise it ends at $v'\vartheta$ with $v' \ne \lambda$. The visit to ϑ can occur at most once by strict weight decrease. Second, since only the top symbol is rewritten, a nonempty stack whose last symbol is $\rceil$ (respectively $\rfloor$) retains that last symbol until the entire stack is popped.

Lemma C.1. *For $0 \le i \le n$, every stack ψ and $w \in \Gamma_{\mathrm{br}}^*$, $E_i\psi \xrightarrow{w} \psi$ if and only if $w \in L_i$. Moreover, if $\sigma \xrightarrow{w} \theta$ and $w \in L_i$, then $\sigma = E_i\psi$ and $\theta = \psi$ for some stack ψ.*

Proof. If $E_i\psi \xrightarrow{w} \psi$, the suffix property forces a first visit to ψ (otherwise the final stack would be $v'\psi$ with $v' \ne \lambda$). The remaining input must be empty by strict weight decrease. Thus this run is exactly a run $E_i \xrightarrow{w} \lambda$ with the suffix ψ retained. Conversely, any such run from E_i lifts to one from $E_i\psi$.

We prove $E_i \xrightarrow{w} \lambda$ iff $w \in L_i$ by induction on i. For $i = 0$, the only possible run is the single $\mathtt{a}$-transition. For $i \ge 1$, a run must start with $\lceil$ or $\lfloor$. In the unary case the stack becomes $E_{i-1}\rceil$. The suffix property forces the child E_{i-1} to be completely processed before the final $\rceil$ is popped. Hence $w = \lceil z \rceil$ with $E_{i-1} \xrightarrow{z} \lambda$, equivalently $z \in L_{i-1}$ by induction. In the binary case the stack becomes $E_{i-1}E_{i-1}\rfloor$. Applying the same suffix property successively to its first and second

E_{i-1} forces $w = \lfloor z_1 z_2 \rfloor$ with $z_1, z_2 \in L_{i-1}$. Both decompositions conversely give accepting runs, proving the equivalence.

For the final assertion, the first i letters of w are openers, followed by $\mathtt{a}$. The first opener (or $\mathtt{a}$ if $i = 0$) requires $\sigma = E_k\psi$. Each opener decreases the exposed index by one, so the following $\mathtt{a}$ requires $k = i$. The established equivalence and determinism give $\theta = \psi$. □

Lemma C.2. *If $0 \le i \le n$ and $w, w' \in L_i$, then $w \equiv_{L_n} w'$.*

Proof. In any run $E_n \xrightarrow{p} \sigma \xrightarrow{w} \theta \xrightarrow{q} \lambda$, Lemma C.1 forces $\sigma = E_i\psi$ and $\theta = \psi$. The same lemma permits $E_i\psi \xrightarrow{w'} \psi$. Thus $(p, q) \in \mathcal{D}_{L_n}(w)$ implies $(p, q) \in \mathcal{D}_{L_n}(w')$; symmetry gives equality. □

Lemma C.3 (branch synchronization)**.** *If $\sigma \xrightarrow{x} \theta$ and $\sigma \xrightarrow{y} \theta$ for nonempty words with different first letters, then $\sigma = E_i\psi$ for some $1 \le i \le n$ and some stack ψ, and there are decompositions $x = wx'$ and $y = w'y'$ with $w, w' \in L_i$ such that $\psi \xrightarrow{x'} \theta$ and $\psi \xrightarrow{y'} \theta$.*

Proof. Only $E_i\psi$ admits two initial moves, reading $\lceil$ and $\lfloor$. Say these occur for x and y, leaving $E_{i-1}\rceil\psi$ and $E_{i-1}E_{i-1}\rfloor\psi$. Until the first visit to ψ, the suffix property and the last-marker property force the respective stacks to have the forms $A\rceil\psi$ and $B\rfloor\psi$. Each then has weight strictly greater than that of ψ. At the first visit to ψ the weight equals that of ψ, and after any further move it is smaller. Thus, if both runs end at the same θ, either both visit ψ or neither does. In the latter case $A\rceil\psi = B\rfloor\psi$ at θ; right cancellation would give $A\rceil = B\rfloor$, impossible because the last markers differ. Both runs therefore visit ψ. Their prefixes up to these visits are in L_i by Lemma C.1, and their remaining suffixes take ψ to θ, as required. □

Proposition C.4. *For arbitrary stacks σ, θ and words $x, y \in \Gamma_{\mathrm{br}}^*$, if $\sigma \xrightarrow{x} \theta$ and $\sigma \xrightarrow{y} \theta$, then $x \equiv_{L_n} y$.*

Proof. Induct on $|x| + |y|$. If $x = \lambda$, then $\sigma = \theta$, so $y = \lambda$ by strict weight decrease; the symmetric case is identical. If $x = cx'$ and $y = cy'$ share their first letter, determinism gives a common state after c, and induction gives $x' \equiv_{L_n} y'$, hence $x \equiv_{L_n} y$. Otherwise Lemma C.3 gives $x = wx'$ and $y = w'y'$ with $w, w' \in L_i$ and $\psi \xrightarrow{x'} \theta$, $\psi \xrightarrow{y'} \theta$. The removed prefixes are nonempty, so induction applies to x', y'. Lemma C.2 gives $w \equiv_{L_n} w'$. The monoid congruence property then gives $wx' \equiv_{L_n} w'y'$, as desired. □

Proof of Proposition 6.3(i). Let $x, y \in \Gamma_{\mathrm{br}}^+$ and $(u, v) \in \mathcal{D}_{L_n}(x) \cap \mathcal{D}_{L_n}(y)$. The two accepting runs from E_n to λ reach the same stack σ after u by determinism. Their stacks θ_x, θ_y before v satisfy $\theta_x \xrightarrow{v} \lambda$ and $\theta_y \xrightarrow{v} \lambda$. Partial injectivity of the word action gives $\theta_x = \theta_y =: \theta$. Thus $\sigma \xrightarrow{x} \theta$ and $\sigma \xrightarrow{y} \theta$. Proposition C.4 gives $\mathcal{D}_{L_n}(x) = \mathcal{D}_{L_n}(y)$. Since L_n is finite and hence context-free, it is Clark–Eyraud substitutable and belongs to $\mathcal{C}^{\mathrm{cf}}_{h_0} \cap \mathcal{C}^{\mathrm{cf}}_{h_{\mathrm{U}}}$. □

References

[1] E. M. Gold. Language identification in the limit. *Information and Control*, 10(5):447–474, 1967.

[2] A. Clark and R. Eyraud. Polynomial identification in the limit of substitutable context-free languages. *Journal of Machine Learning Research*, 8:1725–1745, 2007.

[3] R. Yoshinaka. Identification in the limit of k, l-substitutable context-free languages. In *Grammatical Inference: Algorithms and Applications – ICGI 2008*, LNCS 5278, pp. 266–279, Springer, 2008. doi:10.1007/978-3-540-88009-7_21.

[4] C. de la Higuera. *Grammatical Inference: Learning Automata and Grammars*. Cambridge University Press, 2010.

[5] M. Kanazawa. Identification in the limit of categorial grammars. *Journal of Logic, Language and Information*, 5(2):115–155, 1996. doi:10.1007/BF00173697.

[6] M. Kanazawa. *Learnable Classes of Categorial Grammars*. CSLI Publications, 1998.

[7] T. Kuriyama. *Learning Algorithm for Relation-Substitutable Context-Free Languages*. arXiv:1409.6247v3, November 2014 (historical version). `https://arxiv.org/abs/1409.6247v3`.

[8] R. Yoshinaka. General perspective on distributionally learnable classes. In *Proceedings of the 14th Meeting on the Mathematics of Language (MoL 14)*, pp. 87–98, 2015.

[9] A. Clark and R. Yoshinaka. Distributional learning of context-free and multiple context-free grammars. In J. Heinz and J. M. Sempere (Eds.), *Topics in Grammatical Inference*, pp. 143–172, Springer, 2016. doi:10.1007/978-3-662-48395-4_6.

[10] F. Coste, G. Garet, and J. Nicolas. Locally substitutable languages for enhanced inductive leaps. In *Proceedings of the Eleventh International Conference on Grammatical Inference*, PMLR 21:97–111, 2012.

[11] F. Coste and J. Nicolas. Learning local substitutable context-free languages from positive examples in polynomial time and data by reduction. In *Proceedings of the 14th International Conference on Grammatical Inference*, PMLR 93:155–168, 2019.

[12] F. Coste, D. Fredouille, C. Kermorvant, and C. de la Higuera. Introducing domain and typing bias in automata inference. In *ICGI 2004*, LNCS 3264, pp. 115–126, Springer, 2004. doi:10.1007/978-3-540-30195-0_11.

[13] Y. Takada. Grammatical inference for even linear languages based on control sets. *Information Processing Letters*, 28(4):193–199, 1988. doi:10.1016/0020-0190(88)90208-6.

[14] Y. Takada. Learning formal languages based on control sets. In K. P. Jantke and S. Lange (Eds.), *Algorithmic Learning for Knowledge-Based Systems*, LNAI 961, pp. 316–339, Springer, 1995. doi:10.1007/3-540-60217-8_15.

[15] S. Eilenberg. *Automata, Languages, and Machines*, Vol. A. Academic Press, 1974.

[16] M. Wakatsuki and E. Tomita. A fast algorithm for checking the inclusion for very simple deterministic pushdown automata. *IEICE Transactions on Information and Systems*, E76-D(10):1224–1233, 1993.

[17] R. Eyraud, J. Heinz, and R. Yoshinaka. Efficiency in the identification in the limit learning paradigm. In J. Heinz and J. M. Sempere (Eds.), *Topics in Grammatical Inference*, pp. 25–46, Springer, 2016. doi:10.1007/978-3-662-48395-4_2.

[18] C. de la Higuera. Characteristic sets for polynomial grammatical inference. *Machine Learning*, 27:125–138, 1997.

[19] A. Clark. Learning trees from strings: A strong learning algorithm for some context-free grammars. *Journal of Machine Learning Research*, 14:3537–3559, 2013.

[20] J.-E. Pin. *Mathematical Foundations of Automata Theory*. Lecture notes, version of March 24, 2025. `https://www.irif.fr/~jep/PDF/MPRI/MPRI.pdf`.

[21] J.-E. Pin. Syntactic semigroups. In G. Rozenberg and A. Salomaa (eds.), *Handbook of Formal Languages*, Vol. 1, pp. 679–746, Springer, 1997.

[22] J. C. Costa. Some pseudovariety joins involving locally trivial semigroups. *Semigroup Forum*, 64(1):12–28, 2002. doi:10.1007/s002330010060.

[23] J. C. Costa. Some pseudovariety joins involving groups and locally trivial semigroups. In *Semigroups, Algorithms, Automata and Languages*, pp. 341–348, World Scientific, 2002. doi:10.1142/9789812776884_0013.

[24] S. Ginsburg and E. H. Spanier. Finite-turn pushdown automata. *SIAM Journal on Control*, 4(3):429–453, 1966. doi:10.1137/0304034.

[25] J.-M. Autebert, J. Berstel, and L. Boasson. Context-free languages and pushdown automata. In G. Rozenberg and A. Salomaa (Eds.), *Handbook of Formal Languages*, Vol. 1, pp. 111–174, Springer, 1997.

[26] J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed., Pearson/Addison-Wesley, 2007.

[27] S. Ginsburg and S. A. Greibach. Deterministic context-free languages. *Information and Control*, 9:620–648, 1966.

[28] A. Clark. Distributional learning of some context-free languages with a minimally adequate teacher. In *Proceedings of ICGI 2010*, LNCS 6339, pp. 24–37, Springer, 2010. doi:10.1007/978-3-642-15488-1_4.